Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

First Author[1*] and Second Author[1]

[1*]Department, Organization, City, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;
Contributing authors: second.author@example.com;

Abstract

Sequential approximate Bayesian computation (ABC-SMC/PMC) is the standard tool for likelihood-free inference, but its generation-staged structure imposes synchronization barriers that waste compute when simulator runtimes vary across parameters, precisely the regime of large heterogeneous high-performance computing (HPC) workloads. We introduce a *generation-free, single-arrival-driven* ABC algorithm that combines smooth-kernel ABC with a streaming limit of Adaptive Multiple Importance Sampling (AMIS): the proposal mixture and importance weights update after *every* evaluated particle rather than at population boundaries. The algorithm is *stateless* (a pure function of the evaluated history), which makes it crash-recoverable and a natural fit for the barrier-free island model of the Propulate optimization engine. We show that the *idealized* streaming estimator inherits AMIS's consistency and central-limit guarantees under the Wilkinson smooth-likelihood framework. For efficient execution we additionally provide an optional bounded-memory approximation of the estimator (a fixed sliding-buffer denominator, with a top-$\boldsymbol{k}$ archive as the online proposal); it lies outside the scope of those guarantees, so rather than claim the asymptotics for it we supply direct empirical evidence, via simulation-based calibration of the reported posterior, characterizing where it stays well calibrated and where it does not. Empirically, against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule (and, on all three benchmarks where we report systems gains, against a barrierized twin of our own propagator that isolates synchronization exactly), the method (i) is well calibrated in low dimensions and robust to multimodality — simulation-based calibration of the reported full-history estimator yields near-nominal coverage and the top-$\boldsymbol{k}$ archive retains both modes of a bimodal posterior in $\approx$**86%** of trials — and conservative rather

than over-confident in higher dimensions; and (ii) delivers substantial HPC benefits: under a persistent straggler the synchronous baseline's throughput collapses by more than two orders of magnitude while the asynchronous method holds essentially flat, and under runtime heterogeneity it sustains several-fold higher simulation throughput at a fixed wall-clock budget. On a realistic Cellular Potts tissue-simulation workload the asynchronous method scales almost linearly to eight nodes, sustaining several-fold higher throughput ($\approx$ **5**$\times$ at **384** workers, widening with scale) than a synchronous baseline that, given the same core budget and a matching population, scales only sub-linearly, at a small cost in posterior quality on that benchmark.

Keywords: approximate Bayesian computation, sequential Monte Carlo, adaptive multiple importance sampling, asynchronous algorithms, high-performance computing, likelihood-free inference

1 Introduction

Approximate Bayesian computation (ABC) is a standard tool for simulator-based inference when the likelihood is unavailable or too expensive to evaluate directly [1]. Sequential ABC methods improve efficiency by evolving proposal distributions and tolerance thresholds across a sequence of intermediate targets [2–5]. In practice, however, most ABC-SMC/PMC implementations remain organized around *synchronous* populations: the proposal and tolerance update only after an entire generation has been evaluated. Because simulator runtimes often depend strongly on the sampled parameters, generation barriers leave compute nodes idle waiting for the slowest simulation, a dominant inefficiency on large heterogeneous high-performance computing (HPC) systems.

This paper introduces a generation-free alternative designed for exactly that regime and implemented within the barrier-free island model of the Propulate engine [6]. Our contributions are:

1. A **generation-free, single-arrival-driven ABC algorithm** that combines smooth-kernel ABC [7, 8] with a streaming limit of Adaptive Multiple Importance Sampling (AMIS) [9]. To our knowledge it is the first ABC algorithm whose proposal mixture updates after every evaluated particle rather than at generation/stage boundaries.
2. A **history-reconstructed particle archive** that makes the algorithm *stateless* (a pure function of the evaluated history) and therefore crash-recoverable.
3. **Integration into Propulate**, exploiting its barrier-free island model for true asynchronous execution on HPC.
4. **Asymptotic guarantees for the estimator, with empirical validation of an efficient variant.** We give consistency and a central limit theorem for the *idealized* full-mixture streaming estimator, as a corollary of AMIS theory under the smooth-likelihood framework and with explicit conditions on the proposal sequence and bandwidth schedule (§4). For efficient execution we additionally provide an optional

bounded-memory approximation of this estimator (a fixed sliding-buffer denominator, with a top-k archive as the online proposal); it lies outside the scope of those conditions, so we establish the inference quality of the resulting reported posterior *empirically*, by simulation-based calibration and matched-budget posterior error, rather than by appeal to the asymptotics.

5. An **apples-to-apples empirical evaluation against pyABC** [10, 11] in which pyABC's acceptor is replaced by a probabilistic-rejection acceptor using the *same* ABC kernel and bandwidth schedule. This matched-kernel baseline removes the acceptance rule as a confound, and a **barrierized twin** of our own propagator — identical in every respect but the collective barrier before each breed — isolates synchronization directly on every benchmark where we report a systems gain: 21–402× slower than the asynchronous arm under a persistent straggler (at either of two barrier granularities, which agree to three significant figures wherever the straggler binds, so the gap is not an artifact of how finely the twin synchronizes), 1.1–25× under runtime heterogeneity, and 1.9–4.4× on the Cellular Potts workload, at unchanged posterior quality throughout (§7.1, §7.3); we separately verify (via SBC and matched-budget posterior error) that the reported streaming (full-history AMIS) posterior is comparable to the baseline's in low dimensions and under multimodality, and conservative rather than over-confident in higher dimensions.

The remainder of the paper summarizes the relevant ABC and importance-sampling background (§2), develops the method (§3) and its theory (§4), describes the implementation (§5), lays out the experimental program (§6), and reports results (§7).

2 Background and Related Work

2.1 Approximate Bayesian computation

ABC replaces exact likelihood evaluation with simulation and discrepancy-based acceptance. For observed data y and parameter θ, the ABC target at bandwidth ϵ is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta)\, L_\epsilon(\theta), \qquad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho \mid \theta)}[K_\epsilon(\rho)]\,, \tag{1}$$

where ρ is the discrepancy between simulated and observed summaries, π is the prior, and K_ϵ is the ABC kernel. Classical ABC uses the hard indicator $K_\epsilon(\rho) = \mathbf{1}[\rho < \epsilon]$; *smooth-kernel* ABC [7, 8] replaces this with a continuous kernel (e.g. Gaussian $\exp(-\rho^2/2\epsilon^2)$ or Epanechnikov), which removes the discontinuity in the likelihood approximation and admits standard SMC-sampler theory.

2.2 Sequential ABC algorithms

The generation-staged family (ABC-SMC [2, 3], ABC-PMC [4], and adaptive variants [5, 12, 13]) advances through intermediate targets using mixture proposals

$$q_r(\theta) = \sum_{j=1}^{N_r} W_j^{(r)}\, K(\theta \mid \theta_j^{(r)}) \tag{2}$$

built from accepted particles of the previous stage. The defining constraint is the *generation barrier*: the importance weights at stage r are computed against a single proposal q_{r-1}, which requires the full stage-r population before q_r can be formed.

2.3 Adaptive multiple importance sampling

AMIS [9] generalizes staged importance sampling: at each stage the proposal is adapted from past particles, and crucially *all* past particles are re-weighted against the cumulative proposal mixture $\frac{1}{n}\sum_{j=1}^{n} q_{\tau_j}$ (the balance heuristic of [14]) rather than against the proposal under which each was originally drawn. AMIS is consistent and satisfies a CLT under regularity conditions. Our method takes the *streaming limit* of AMIS: stage size one, single-arrival-driven adaptation.

2.4 Asynchronous SMC and parallel ABC

Our proposal-adaptation machinery descends from the adaptive-importance-sampling line — population Monte Carlo [15], AMIS [9] and its recycling-consistency theory [16], surveyed in [17] — which we take to its streaming, single-arrival limit. Parallel and off-barrier SMC has been studied for state-space models — the island particle model [18], asynchronous anytime particle methods [19], and parallel resampling [20] — but none targets ABC, and none combines barrier-free execution with AMIS-style cumulative-mixture reweighting. Distributed ABC frameworks such as pyABC [10, 11], ABCpy [21], and ELFI [22] parallelize *simulation* effectively but retain population-level synchronization: workers stall while in-flight simulations drain at the generation boundary. Our contribution removes the barrier entirely; the trade-off is that standard generational SMC theory no longer applies, which motivates §4.

3 Method

3.1 Design constraints and history-based state

We design the algorithm to be *stateless*: every quantity that defines the reported estimator and the archive is a pure function of the evaluated history. Statelessness is a desirable property in its own right, as it makes the reported estimator reproducible up to MPI arrival order and crash-recoverable, each candidate being reconstructible from the recorded history alone; the only carried state — the online AMIS snapshot buffer and two scheduler throttles — is performance state, rebuilt empty after a restart, which the retroactive estimator supersedes. For an efficient implementation we leverage the existing Propulate framework, whose propagator exposes a single call `__call__(inds) -> Individual`: it receives the entire evaluated history and emits the next candidate without persistent archive state or assimilation callbacks, exactly the stateless, history-based interface our design calls for. All algorithmic state is thus reconstructed from history. Define the evaluated history at call n as

$$\mathcal{H}_n = \left\{(\theta_i, \rho_i, \epsilon_i, \tau_i, w_i)\right\}_{i=1}^{n}, \tag{3}$$

where θ_i is the parameter, ρ_i the discrepancy, ϵ_i the bandwidth active when θ_i was proposed, τ_i the proposal-time index, and w_i the stored core importance weight π/q_{τ_i}.

3.2 Tolerance reconstruction and monotonicity

Each proposed individual stores the bandwidth active at proposal time. The effective bandwidth reconstructed from history is the running minimum of these stored bandwidths, $\epsilon_{\text{hist}} = \min_i \epsilon_i$ (falling back to the initial tolerance on an empty history). The scheduler proposes ϵ_{sched}, and the bandwidth used is $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$, giving a monotone-decrease guarantee with no mutable propagator state.

3.3 Archive and smooth-kernel proposal

The reconstructed archive is the top-k history members by lowest discrepancy, $A_n = \text{Top}_k(\{\theta_i\}, \text{order by } \rho_i)$. The proposal mixture uses kernel-weighted archive members,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_\Sigma(\theta - \theta_j), \qquad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \tag{4}$$

with $\sum_j \tilde{W}_j^{(n)} = 1$. The perturbation kernel K_Σ is Gaussian with covariance $\Sigma_n = s\,\widehat{\text{Cov}}_{\tilde{W}}(A_n)$ estimated from the kernel-weighted archive (s is the `perturbation_scale` hyperparameter), factored once via Cholesky. Smooth kernels remove the prior-vs-archive discontinuity present in hard-threshold ABC-PMC: every evaluated particle contributes continuously through its kernel weight.

3.4 Streaming AMIS importance weight

A particle $\theta^\star$ drawn from q_n receives an importance weight under the balance heuristic over a sliding ring buffer $\mathcal{S}$ of past proposals (size S, sampled every `amis_interval` calls):

$$w^\star = \frac{\pi(\theta^\star)}{\bar{q}_n(\theta^\star)}, \qquad \bar{q}_n(\theta) = \frac{1}{1 + |\mathcal{S}|}\Big[q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta)\Big]. \tag{5}$$

Setting $|\mathcal{S}| = 0$ recovers single-current-proposal weighting; increasing S enriches the balance-heuristic denominator toward the full cumulative mixture, which lowers the variance of the importance weights at a modest additional per-call cost. We use $S = 20$ by default, a value that captures most of that variance reduction while keeping each update cheap. A particle's weight, assigned at proposal time, is corrected by the cumulative-mixture denominator at every subsequent use (the retroactive AMIS step). Because each snapshot q_s is the proposal reconstructed from the history truncated at call s, the buffer $\mathcal{S}$ is itself a deterministic function of $\mathcal{H}_n$, so the propagator stores nothing between calls. Equation (5) is the proposal-time importance weight; the *posterior* estimator reported in §4 reweights it by the smooth ABC likelihood $K_{\epsilon_n}(\rho)$ (6).

Three weight objects.

To avoid conflation, the method uses three distinct weightings. (i) The *adaptation weights* $\tilde{W}_j^{(n)}$ (4) are kernel weights over the top-k archive; they define the next proposal mixture q_n and so drive proposal *adaptation* only. (ii) The *proposal-time importance weight* $w^\star$ (5) is the balance-heuristic weight over the bounded sliding buffer $\mathcal{S}$ ($S = 20$), assigned online; it enters proposal adaptation through (4) and the effective-sample-size diagnostic, but never the reported posterior. (iii) The *reported posterior* is the retroactive estimator $\widehat{\pi}_n$ (6), in which every evaluated particle is reweighted by $\pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i)$, where $\bar{q}_n$ is a draw-proportional mixture of $m \leq S = 20$ history-reconstructed proposals plus a defensive prior component (Appendix A) — the Condition 4 approximation to the full cumulative mixture, not the mixture over every evaluated particle. The reported posterior densities and the SBC calibration are computed from it; the Wasserstein-vs-time quality curves are computed on the top-k archive (asynchronous) and the final population (synchronous), matched in size.

Stored fields and weight lifecycle.

Each evaluated individual i records exactly four fields: its parameter θ_i, its discrepancy ρ_i, the bandwidth ϵ_{τ_i} active when it was proposed, and its proposal-time weight $w_i^\star$ (5); the history $\mathcal{H}_n$ is the arrival-ordered sequence of these records. From $\mathcal{H}_n$ alone: the archive is the top-k members by smallest ρ under the reconstructed running-minimum bandwidth; the adaptation weights $\tilde{W}_j^{(n)}$ (weight (i)) are *recomputed* each call from the current archive (never stored) and define q_n; the proposal-time weight $w_i^\star$ (weight (ii)) is computed once at proposal time over the sliding buffer $\mathcal{S}$, stored, and consumed by the ESS diagnostic and by parent selection during adaptation; and the reported posterior weight (weight (iii)) is *recomputed* retroactively from $\mathcal{H}_n$ using the $m \leq S$ snapshot denominator with a defensive prior floor. Only $w_i^\star$ is persisted; weights (i) and (iii) are pure functions of $\mathcal{H}_n$ recomputed on demand, so after a crash the archive and the reported estimator are reconstructed from the recorded history, while the online buffer and the two scheduler throttles rebuild empty and affect only the proposal-time diagnostic.

3.5 Update loop

Algorithm 1 assembles the pieces above into the single-arrival update the propagator runs on each call. Every quantity it uses (the effective bandwidth, the archive, the proposal mixture, and the AMIS denominator) is reconstructed from the history $\mathcal{H}_n$ passed in, so the update carries no state of its own between calls; until the history holds at least k members it emits uniform prior draws to seed the archive.

3.6 Relation to classical ABC-SMC

Table 1 sets the method against a typical generation-staged ABC-SMC/PMC implementation — concretely the pyABC baseline used in our experiments — property by property. Such an implementation advances a fixed population through discrete generations behind a synchronization barrier, weights each stage against the single previous

Algorithm 1 History-reconstructed, single-arrival-driven ABC update

Require: History $\mathcal{H}_n$, archive size k, initial bandwidth ϵ_0, snapshot buffer $\mathcal{S}$

Reconstruct ϵ_{hist} from stored tolerances; compute ϵ_{sched}; set $\epsilon_n \leftarrow \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$
if $|\mathcal{H}_n| < k$ **then**
    emit a uniform prior draw (bootstrap) ▷ covers $\bar{q}$ on the whole box
else
    select archive A_n (top-k by ρ); form $\tilde{W}^{(n)}$ in log-space via K_{ϵ_n}
    build Σ_n, Cholesky-factor once; sample $\theta^\star$ by perturbing a $\tilde{W}^{(n)}$-weighted parent
    compute $w^\star$ via the AMIS denominator (5); periodically snapshot q_n into $\mathcal{S}$
end if
return $\theta^\star$ for external simulation and discrepancy evaluation

Table 1 A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize the configured baseline, not ABC-SMC as a family.

Property	Classical ABC-SMC	This work
Update style	generation, barrier-synchronized	event-driven (single-arrival), no barrier
Archive	explicit, generational	reconstructed, sliding top-k
ABC likelihood	hard threshold	smooth kernel
Importance weight	against q_{t-1} only	balance heuristic over $\mathcal{S}$
State	mutable	stateless (function of history)

proposal, and carries an explicit mutable archive; none of these is intrinsic to ABC-SMC in general (smooth kernels and richer proposals exist), but each characterizes the synchronous baseline we compare against. The proposed method replaces each of these with a barrier-free counterpart: a single-arrival update in place of the generation step, a sliding top-k archive reconstructed from history in place of the explicit population, a smooth ABC kernel in place of the hard threshold, and a balance-heuristic weight over the cumulative mixture in place of single-proposal weighting. The decisive difference is the last row: because all state is a pure function of the history, there is nothing to synchronize between workers, which is what makes the method a natural fit for the asynchronous island model of §5.

4 Theoretical Analysis

We state consistency and a CLT for the *idealized* full-mixture streaming estimator, inheriting the consistency theory for AMIS-type recycling schemes [16] under the smooth-likelihood ABC framework [7]; AMIS itself was introduced by [9], whose original convergence argument is heuristic. The streaming regime (stage size one) is a degenerate case of AMIS stages. Below, $\bar{q}_n$ denotes the snapshot denominator actually evaluated and $\frac{1}{n}\sum_{j\le n} q_{\tau_j}$ the idealized full cumulative mixture; Condition 4 bridges

them. The estimator is

$$\widehat{\pi}_n(\theta) = \frac{\sum_{i=1}^n w_i^{\mathrm{bal}}(n)\,\delta_{\theta_i}(\theta)}{\sum_{i=1}^n w_i^{\mathrm{bal}}(n)}, \qquad w_i^{\mathrm{bal}}(n) = \frac{\pi(\theta_i)\,K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}. \tag{6}$$

Condition 1 (bounded proposal-density ratio)**.** *There exist* $c, C > 0$ *with* $c \le q_\tau(\theta)/\pi(\theta) \le C$ *for every generated proposal* q_τ *and every* $\theta \in \operatorname{supp}\pi$*.*

Condition 2 (kernel regularity)**.** K_ϵ *is non-negative, integrable, peak-normalized* ($K_\epsilon(0) = 1$*, unit* peak*, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and Lipschitz in* ϵ *for fixed* ρ*.*

Condition 3 (bandwidth schedule)**.** $\{\epsilon_n\}$ *is monotone non-increasing with* $\epsilon_n - \epsilon_{n+1} \to 0$ *and* $\epsilon_n \downarrow \epsilon_\infty > 0$*. For the CLT we additionally require the schedule to be fast enough that the residual bias vanishes at the Monte Carlo scale,* $\sqrt{n}\,\{\pi_{\epsilon_n}(f) - \pi_{\epsilon_\infty}(f)\} \to 0$ *(which monotone convergence alone does not imply; a schedule as slow as* $1/\log n$ *violates it).*

Condition 4 (snapshot approximation)**.** $\sup_\theta \left|\bar{q}_n(\theta) - \frac{1}{n}\sum_{j=1}^n q_{\tau_j}(\theta)\right| \to 0$ *as* $n \to \infty$*.*

Theorem 1 (Consistency)**.** *Under Conditions 1–4, for every continuous* π_{ϵ_∞}*-integrable* f*,* $\int f\, d\widehat{\pi}_n \xrightarrow{a.s.} \int f\, d\pi_{\epsilon_\infty}$*.*

Theorem 2 (CLT)**.** *Under Conditions 1–4 and a finite second moment of* $w_i^{\mathrm{bal}} f(\theta_i)$*,* $\sqrt{n}\big(\int f\, d\widehat{\pi}_n - \int f\, d\pi_{\epsilon_\infty}\big) \xrightarrow{d} \mathcal{N}(0, \sigma_f^2)$ *with* σ_f^2 *the AMIS asymptotic variance at the streaming limit.*

Proof sketch. Each draw is a pair (θ_i, ρ_i) with $\theta_i \sim q_{\tau_i}$ and $\rho_i \sim p(\cdot \mid \theta_i)$; the balance-heuristic weight uses the bounded kernel $K_{\epsilon_n}(\rho_i)$, so the estimator is importance sampling on the augmented space (θ, ρ) against the smooth-ABC target $\propto \pi(\theta)K_{\epsilon_n}(\rho)$ [7]. Conditions 1–2 bound the conditional weight variance; 3 makes $\pi_{\epsilon_n} \Rightarrow \pi_{\epsilon_\infty}$ (and, with its CLT clause, controls the residual bias at the $\sqrt{n}$ scale); 4 makes the evaluated denominator $\bar{q}_n$ converge to the idealized cumulative mixture. Consistency then follows from the consistency theory for AMIS-type recycling schemes [16] applied to the streaming limit (their scheme restricts adaptation so that the weighting stage decouples; we invoke it for the idealized full-mixture estimator). The CLT yields confidence intervals for posterior expectations at the idealized limit; classical SMC/ABC-PMC central-limit theory is itself well developed [5], though not routinely exposed per run in the generational implementations we compare against. Limitations are discussed in §9.

Theory versus the efficient implementation.

Theorems 1–2 characterize the *idealized* streaming estimator that reweights against the full cumulative mixture $\frac{1}{n}\sum_{j\le n} q_{\tau_j}$. The *reported* posterior evaluates $\widehat{\pi}_n$ with the bounded snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \le S = 20$

history-reconstructed proposals plus a defensive prior component (mass floored at $0.5/(m{+}1)$) — not the exact full mixture; Condition 4 is precisely the assumption that $\bar{q}_n$ tracks the cumulative denominator, credible once the proposal sequence has stabilized but not guaranteed. This makes two departures from the idealized object. First, the reported (and the online proposal-time) denominator uses $m \leq S = 20$ reconstructed proposals rather than all n, so the retroactive pass costs $\mathcal{O}(n\,m\,k)$ with m fixed — i.e. $\mathcal{O}(nk)$ — not the $\mathcal{O}(n^2k)$ of a literal full mixture (Appendix A). Second, Condition 1 (a two-sided density-ratio bound relative to the prior) does not hold for a bare top-k local Gaussian proposal across the whole support; the defensive prior component and the uniform prior-draw bootstrap phase (Alg. 1) supply only partial coverage, so formalizing the estimator with a persistent defensive mixture $\delta\pi{+}(1{-}\delta)q_n$ and a complete CLT with explicit rate is left to future work. We therefore read §4 as the asymptotic target the method is designed around, and verify the calibration of the estimator *as actually implemented* directly by simulation-based calibration (§7.2).

5 Implementation in Propulate

The algorithm is a stateless Propulate propagator: tolerance schedulers are pure functions of history; the archive is the top-k history members by smallest discrepancy under the reconstructed bandwidth; the smooth-kernel mixture weights are formed in log-space for numerical stability. The reported posterior is the retroactive AMIS estimator (6), computed off the timed inference path from the saved history (a pure function of it, hence identical whether computed online or after a crash/restart); the bare proposal-time weight (5) is retained separately for the effective-sample-size diagnostic. This retroactive step is a one-time pass of cost $\mathcal{O}(nk)$ in the n evaluated particles and archive size k, run after the timed budget and therefore excluded from the throughput measurements by design (the systems comparison is over simulation throughput). For cost-bearing simulators it is negligible against the simulation time; for near-instantaneous simulators whose history reaches $\mathcal{O}(10^7)$ particles it is non-trivial but bounded ($\mathcal{O}(nk)$ with the fixed $m \leq S = 20$ snapshot count), and is evaluated in fixed-memory chunks (Appendix A) so it stays a bounded fraction of wall-clock and memory. For an apples-to-apples comparison, pyABC's `UniformAcceptor` is replaced by a probabilistic-rejection acceptor using the same $K_\epsilon(\rho)$ and bandwidth schedule as the asynchronous side. The acceptor and kernel are thus matched across methods; the synchronous baseline differs in its generation-staged *execution* (the systems variable under test) and, on the inference side, in retaining classical population weighting rather than the streaming AMIS estimator, whose quality we check separately (§7.2). Distributed execution uses Propulate's barrier-free island model over MPI; for wall-time-limited HPC experiments the synchronous baseline uses fixed populations/generations, which yields more interpretable comparisons than contrasting methods that stop under different ϵ rules.

Table 2 Benchmark suite and role in the evaluation.

Benchmark	Role	Outputs
Gaussian mean	sanity / calibration	analytic-posterior error, SBC coverage
g-and-k	intractable-likelihood	posterior recovery, Wasserstein vs. budget
Lotka–Volterra	simulator-based ABC	posterior recovery, quality vs. time
Cellular Potts	realistic HPC workload	throughput, utilization, scaling, posterior

6 Experimental Design

6.1 Research questions

(RQ1) Does the method recover posteriors comparable to established ABC baselines? **(RQ2)** Does asynchronous execution improve wall-clock efficiency and utilization under runtime heterogeneity? **(RQ3)** Does it scale on a realistic, expensive simulator, the regime where running a synchronous matched baseline would itself be prohibitively wasteful?

6.2 Benchmarks, baselines, and metrics

We use four benchmarks of increasing realism (Table 2): Gaussian-mean inference (analytic posterior; sanity and calibration), the g-and-k distribution (classical intractable-likelihood benchmark), the stochastic Lotka–Volterra system, and a Cellular Potts tissue model (`cellsInSilico`) as the realistic heterogeneous-runtime workload. Baselines are rejection ABC (small problems), the matched-kernel synchronous baseline (a probabilistic-rejection pyABC variant, the main comparator), and pyABC as an external reference. Statistical metrics: posterior mean error, (sliced) Wasserstein distance to the truth, and credible-interval coverage via simulation-based calibration (SBC). Computational metrics: simulation throughput, worker utilization / idle fraction, and wall-clock time to a fixed posterior-error target. Convergence curves are placed on a shared time grid via last-observation-carried-forward resampling so asynchronous (fine-grained) and synchronous (per-generation) records compare fairly.

6.3 HPC studies

Three dedicated studies probe the asynchrony claim: (i) *stochastic runtime heterogeneity* (a log-normal post-evaluation delay with sweep over spread σ); (ii) *straggler tolerance* (one worker given a permanent post-evaluation delay scaled by $0\times$ (control), $1\times, 5\times, 10\times, 20\times$); and (iii) *strong scaling* at fixed wall-clock budget over worker counts from 1 up to 288 (six fully-packed nodes) on Lotka–Volterra and up to 384 (eight nodes) on Cellular Potts, with up to five replicates each. A hyperparameter *sensitivity* grid (archive size, perturbation scale, tolerance scheduler, initial tolerance) and an *ablation* (hard vs. smooth kernel; with vs. without AMIS reweighting) complete the suite.

7 Results

Unless noted, results are five-replicate medians on JUWELS; figures compare the matched-kernel synchronous baseline (labelled *Synchronous*) against the proposed asynchronous method (*Asynchronous*, ours). We lead with the computational evidence (RQ2), the paper's central claim, then verify that inference quality is not sacrificed (RQ1), and close with the realistic workload (RQ3).

7.1 Computational advantage (RQ2)

Straggler robustness.

This is the clearest illustration of the barrier's cost (Fig. 1). One worker is given a permanent post-evaluation delay of 0.1 s scaled by $0\times$ (control), $1\times, 5\times, 10\times, 20\times$. As the injected delay grows from the control to $20\times$, the synchronous baseline's throughput collapses by more than two orders of magnitude (median $\approx 8800 \to 60$ simulations/s) because each generation blocks on the straggler, whereas the asynchronous method routes work to available workers and holds essentially flat ($\approx 3900 \to 3800$ simulations/s); only at the $0\times$ control is the baseline faster, where a generation barrier is cheap for this near-instant simulator. Posterior quality is comparable throughout (final Wasserstein-to-truth ≈ 0.07–0.08 for both methods; §7.2).

Isolating synchronization: a barrierized twin.

The comparison above is against a different algorithm, so it cannot by itself attribute the gain to the barrier (§9, item vii). We therefore build a *barrierized twin* of our own method: the identical propagator — same proposals, same kernel weights, same archive rule, same reported estimator — with a collective barrier inserted immediately before each breed, so that the asynchronous arm and the twin differ in *nothing but* when a worker is permitted to proceed. Table 3 reports both on the straggler benchmark over five replicates. Only the twin needs a simulation-count budget: a collective barrier requires identical per-rank call counts, so a wall-clock stop — which halts ranks independently, as the first one to reach the deadline — deadlocks it. The asynchronous arm therefore runs wall-limited (300 s of active wall-clock) and the twin runs to a fixed simulation count, and we compare the two as *rates*: simulations per second of active wall-clock on both arms. The asynchronous arm holds ≈ 3100–3900 simulations/s at every slowdown, reproducing the throughput of the main experiment above; the twin runs at 48.7 simulations/s already at the $0\times$ control — the cost of synchronizing a ≈ 4 ms simulator, where barrier latency alone dominates — and degrades to 8.0 as the straggler worsens, a factor of 402 below the asynchronous arm at $20\times$. Final Wasserstein-to-truth is 0.07–0.08 on *both* arms at every slowdown, confirming that the twin changes the schedule and not the statistics. Barrier removal is therefore worth between one and two-and-a-half orders of magnitude of throughput on this workload at unchanged posterior quality, measured against our own algorithm rather than inferred from a cross-algorithm comparison.

Is the twin's fine generation doing the work? The twin above synchronizes every $W{=}16$ evaluations, one per worker — a finer generation than the synchronous baseline's population of 100 — so one could suspect the ratio is an artifact of barrier

frequency rather than of synchronization as such. We tested this directly by re-running the twin with the barrier coarsened to one per 112 evaluations ($7W$, matching the baseline's population), five replicates at every slowdown (Table 3, third row). Granularity turns out to matter only where the straggler does not: at the 0× and 1× controls the coarse twin is 3.3× and 2.3× faster (48.7 → 162.8 and 65.3 → 148.4 simulations/s), cutting the asynchronous lead from 80× to 24× and from 48× to 21×; but at 5×, 10× and 20× the two granularities agree to three significant figures (31.6 vs 31.8, 15.91 vs 15.94, 7.98 vs 7.98), leaving the ratios at 102–402×. The mechanism is straightforward: a barrier costs either its own latency or the wait for the slowest member of the generation, whichever is larger. Coarsening amortizes the latency over more evaluations, which helps when latency dominates, but every generation still has to wait for the straggler no matter how long the generation is — so in the straggler regime the cost is granularity-independent. The advantage we report under a straggler is thus not an artifact of the twin's schedule, and the remaining caveat is one of workload rather than granularity: this is a deliberately adversarial near-instant simulator, and where per-evaluation cost amortizes the coordination (§7.3) the barrier's relative cost is correspondingly smaller.

The same test under runtime heterogeneity.

A persistent straggler is the sharpest case but not the typical one, so we repeat the twin comparison on the heterogeneity benchmark, where every worker draws a fresh lognormal runtime multiplier at every evaluation rather than one rank being permanently slow. Both barrier granularities again, five replicates at each of the five heterogeneity levels, simulation-limited with the budget per level sized from measured rates (Table 4). The picture differs from the straggler case in two instructive ways. First, the σ=0 control now costs essentially *nothing*: the twin runs at 44.8 simulations/s against the asynchronous arm's 47.6, a factor of 1.1, where on the straggler benchmark the same control already cost 24–80×. The difference is the simulator, not the barrier — these evaluations take ≈1 s rather than ≈4 ms, so barrier latency is negligible against the work it separates. The barrier's cost on a cost-bearing simulator is therefore a pure *variance* cost, and it appears only once there is variance to pay for: it grows monotonically to 10.1× at σ=1.5 and 25.3× at σ=2 at the baseline-matched granularity (30.9× at the finer one). Second, coarsening the barrier now *does* help — 1.2 to 1.7× across $\sigma \geq 0.5$, against the 1.00× it bought under the persistent straggler. That contrast is the mechanism made visible: coarsening trades a heavier per-barrier maximum against half as many barriers, which is a favourable trade when the slow draw is transient and a different worker each time, and no trade at all when the same rank is slow in every generation. Posterior quality is again unaffected — the twin recovers the analytic posterior mean to within 0.003–0.034 absolute error against the asynchronous arm's 0.007–0.011, with the largest twin value at σ=2, where its budget is only ten generations.

There is one further reading worth stating, because it runs opposite to the concern the twin was built to address. The worry about a cross-algorithm comparison is that it might *overstate* the barrier's role. Here it understates it: the synchronous pyABC baseline's penalty grows only from 2.1× to 4.8× over the same heterogeneity range, so

Table 3 Barrierized twin on the straggler benchmark: median simulation throughput (simulations per second of active wall-clock) over five replicates. The twin is the *same* propagator with a collective barrier before each breed, so the rows differ only in synchronization. Two barrier granularities are shown: one barrier per W=16 evaluations (one per worker, the finest possible generation) and one per 112 evaluations ($7W$, matching the synchronous baseline's population of 100). Coarsening the generation to the baseline's costs the twin nothing once the straggler binds — the 5×/10×/20× columns agree to three significant figures — and buys a factor of 3.3/2.3 only at the 0×/1× controls, where barrier latency rather than straggler waiting is the bottleneck. Final Wasserstein-to-truth is 0.07–0.08 on all three arms at every slowdown.

Straggler slowdown	0×	1×	5×	10×	20×
Asynchronous (ours)	3888	3112	3241	3218	3205
Barrierized twin, every W=16	48.7	65.3	31.6	15.9	8.0
Barrierized twin, every 112	162.8	148.4	31.8	15.9	8.0
Ratio, twin every W	80×	48×	103×	202×	402×
Ratio, twin every 112	24×	21×	102×	202×	402×

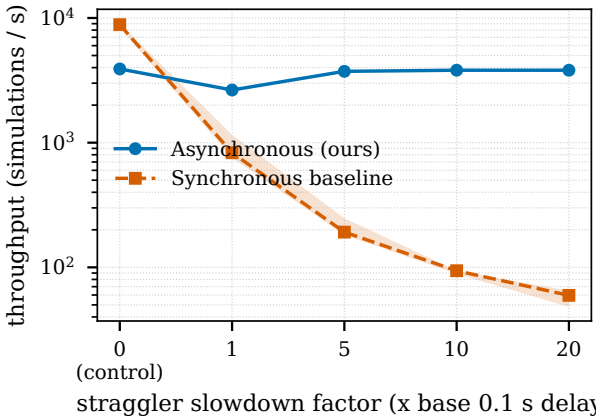

Fig. 1 Throughput (median + inter-quartile range over five replicates) versus straggler slowdown factor, including a 0× no-straggler control. One worker carries a permanent post-evaluation delay of 0.1 s scaled by the factor. The synchronous baseline degrades sharply at the generation barrier (from ≈8800 sims/s at the control to ≈60 at 20×); the asynchronous method is robust (median ≈2600–3900 across the range, with no collapse). Note the logarithmic throughput axis.

at σ=2 our own barrierized twin is 5.3× slower than the external baseline at matched barrier granularity. The design difference that accounts for it is the one noted in §9, item iv: pyABC closes a generation on the first 100 evaluations to finish and discards the latecomers, so it never waits for the slowest draw, whereas the twin — like our own propagator, minus asynchrony — keeps every evaluation and therefore pays the full maximum. The barrier's unmitigated cost is thus larger than the measured gap over pyABC, and the headline comparison is conservative rather than flattering.

Table 4 Barrierized twin under *runtime heterogeneity* (each worker draws a fresh lognormal runtime multiplier of scale σ at every evaluation), 48 workers, median throughput over five replicates. Same two barrier granularities as Table 3: one per W=48 evaluations and one per 96, the latter matching the synchronous baseline's population of 100. Unlike the straggler benchmark, the σ=0 control costs almost nothing (1.1×) because these evaluations take ≈1 s and barrier latency is negligible against them — the cost here is purely the wait for the slowest draw, and it grows with σ. Coarsening the barrier helps under transient heterogeneity (1.2–1.7×), where under a persistent straggler it bought nothing. The synchronous pyABC baseline degrades far less than our own twin because it closes a generation on the first 100 finishers and discards latecomers, so at σ=2 the twin is 5.3× slower than the external baseline it stands in for. All rates are simulations per second of each arm's own elapsed wall-clock; at σ=1.5 and 2 the baseline's runs end at ≈54 and ≈52 s of the 60 s budget, so dividing by its actual elapsed time credits it with the higher rate and makes the comparison against it conservative.

Heterogeneity σ	0	0.5	1.0	1.5	2.0
Asynchronous (ours)	47.6	42.1	28.9	17.6	11.4
Barrierized twin, every W=48	44.8	15.1	4.37	1.06	0.37
Barrierized twin, every 96	45.1	19.8	6.50	1.75	0.45
Synchronous pyABC baseline	23.1	18.9	9.91	4.35	2.39
Ratio, twin every W	1.1×	2.8×	6.6×	16.6×	30.9×
Ratio, twin every 96	1.1×	2.1×	4.4×	10.1×	25.3×
Ratio, pyABC baseline	2.1×	2.2×	2.9×	4.1×	4.8×

Runtime heterogeneity.

Under stochastic log-normal runtime noise the asynchronous method sustains higher utilization (lower idle fraction) than the synchronous baseline, which idles at generation barriers (Fig. 2). Crucially, this utilization converts directly into inference quality: at a fixed wall-clock budget the synchronous baseline completes fewer and fewer simulations as the runtime spread σ grows (it idles at every barrier) and its posterior-mean error to the analytic Gaussian posterior climbs several-fold (from ≈0.05 to ≈0.28), whereas the asynchronous method completes several-fold more simulations and holds its error essentially flat (≈0.007–0.011) (Fig. 3). Removing the barrier therefore preserves not just the simulation rate but the *inference* delivered per unit wall-clock.

Parameter-coupled runtime (every-particle bias).

The study above slows simulations independently of their parameters; a sharper test couples runtime *to the inferred parameter*, directly probing the every-particle bias of Limitation (iv). On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) we set the post-evaluation delay to $d(\mu) = \min\big(d_0\, e^{\,c(\mu-\mu_c)/\ell},\, d_{\max}\big)$ with base $d_0 = 0.02$ s, centre $\mu_c = 0$, length scale $\ell = 0.15$, and cap $d_{\max} = 0.5$ s, and sweep the coupling strength $c \in \{0, 0.5, 1, 2\}$ ($c = 0$ recovers uniform runtime). Larger c makes

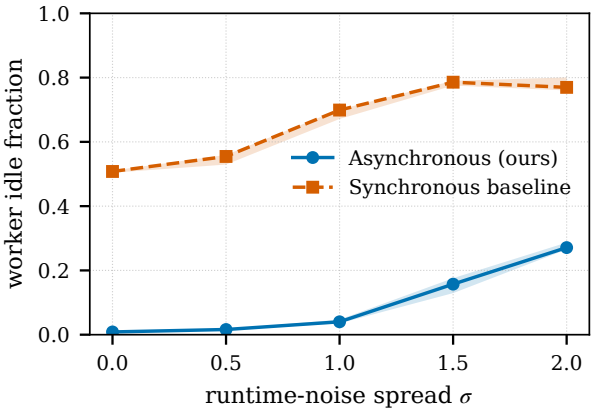

Fig. 2 Worker utilization loss under runtime heterogeneity: the fraction of worker wall-clock spent idle versus the runtime-noise spread σ (five replicates; bands are inter-quartile ranges). The synchronous baseline idles at generation barriers (its idle fraction rises from ≈0.51 to ≈0.79 as heterogeneity grows) while the asynchronous method stays near-fully utilized (idle fraction ≈0.01 rising to ≈0.27).

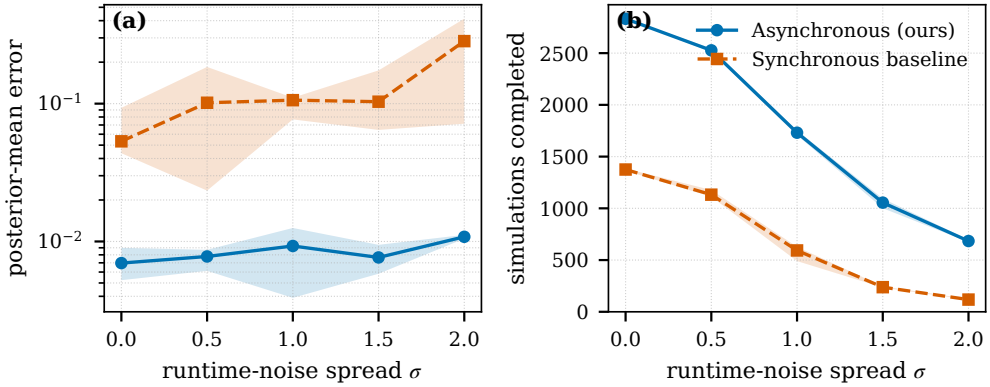

Fig. 3 Inferential efficiency under runtime heterogeneity (Gaussian-mean benchmark, analytic posterior, fixed 60 s budget, five replicates; bands are inter-quartile ranges). Left: posterior-mean error to the analytic posterior versus runtime-noise spread σ; the synchronous baseline degrades sharply while the asynchronous method stays flat. Right: simulations completed within the budget; the synchronous baseline is starved by barrier idling as σ grows, whereas the asynchronous method sustains several-fold higher completion.

one region of parameter space evaluate far faster than its mirror image, so the asynchronous "keep every accepted particle" rule over-represents the fast region in the raw archive. The coupling has the intended effect: aggregate throughput falls as it steepens, by ≈40% for the asynchronous method (≈1880 → 1150 sims/s) and more steeply for the barrier-bound baseline (≈840 → 250; Fig. 4a), confirming that the fast region dominates the raw sample counts. Yet the asynchronous posterior-mean error to the analytic Gaussian posterior does not increase as the coupling steepens; it stays ≈0.01

and comparable to the discard-latecomers synchronous baseline (Fig. 4b). At the coupling levels tested the over-representation is measurable in the raw sampling rate but does not measurably distort the reported (unweighted top-k) posterior mean. A targeted ablation that removes the AMIS proposal adaptation (S=0) and re-runs the same coupling sweep leaves this robustness essentially unchanged — the asynchronous posterior-mean error stays ≈0.005–0.013 with or without AMIS and shows no trend in the coupling — which indicates that the raw archive is itself robust at these levels rather than that AMIS actively corrects the over-representation. Two finer tests that Limitation (iv) motivates sharpen this rather than leave it open. First, we score the AMIS-*reweighted* posterior — the estimator the guarantees of §4 concern, resampling the archive by the retroactive weight (6) — instead of the unweighted archive, and compare it against the no-AMIS variant (S=0) across the coupling sweep and a steeper regime (base delay 0.5 s, cap 8 s). The reweighted posterior recovers the analytic mean as accurately as the unweighted archive (absolute error ≤0.025, no trend in the coupling) in both regimes and with or without AMIS; the reweighting's effect surfaces in the posterior *spread*, not the mean — it lowers the effective sample size (ESS fraction ≈0.88, vs ≈0.99 without AMIS), roughly triples the maximum normalised weight (≈0.03 vs ≈0.01), and widens the central 90% interval by ≈15% — so AMIS reshapes the tails but neither creates nor removes a mean bias at these coupling levels. Second, a drain-after-deadline variant lets every in-flight simulation finish past the wall-clock deadline and recomputes the posterior; comparing it against the censored run isolates the completion-time censoring that dividing by the proposal density does not correct. Draining moves the weighted posterior mean by at most 0.013 — comparable to the replicate-to-replicate spread (≈0.01) — with no systematic direction, in both the mild and the steep regime. At these coupling levels the deadline censoring is therefore present in principle but negligible in effect, consistent with the sampler concentrating in the fast region near the posterior where few simulations are in flight at the deadline. We accordingly report the raw archive as robust here rather than claiming AMIS corrects the over-representation, and treat a formal censoring correction as future work (Limitation (iv)).

Strong scaling.

Lotka–Volterra is deliberately an *adversarial* simulator for a scaling study: each evaluation takes only ≈ 2 ms, so the simulator occupies only a small fraction of each worker's wall-clock and there is essentially no per-evaluation compute to distribute. A dedicated instrumented run measures this productive fraction directly (Fig. 6): at the 48-worker throughput peak a worker spends only ≈25% of its time inside the simulator, leaving ≈75% for coordination and idle waiting, and that productive share falls to ≈8% and then ≈2% as the job spreads from one node to three to six. The mechanism is the method's single shared population: every completed evaluation is broadcast to all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. On a near-instantaneous simulator this exchange saturates and then actively slows the productive ranks. Aggregate throughput therefore rises almost linearly while the job fits on one node (149 → 369 → 1983 → 6240 sims/s at 1 → 4 → 16 → 48 workers) and then

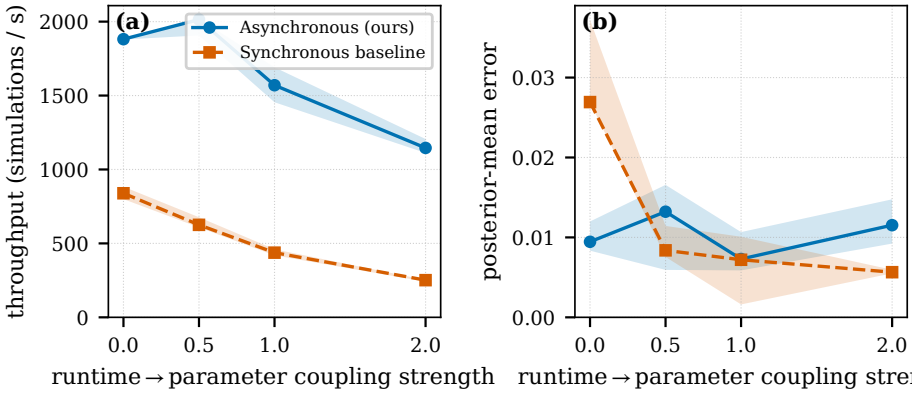

Fig. 4 Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens (more steeply for the barrier-bound baseline), confirming that the fast region dominates the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous method and the discard-latecomers synchronous baseline, so the over-representation does not measurably distort the reported posterior at these coupling levels.

declines across fully packed multi-node jobs (worker counts that are exact multiples of 48): 6240 → 4195 → 2615 → 2088 → 1729 sims/s at 48 → 144 → 192 → 240 → 288 workers, i.e. 1 → 3 → 4 → 5 → 6 nodes, because adding ranks beyond one node lengthens the all-to-all exchange faster than it adds useful work. This is a property of coordinating a near-instantaneous simulator across a single shared population, not of asynchrony as such (precisely the regime in which asynchrony has least to offer), and it lifts once the simulator carries real per-evaluation cost that amortizes the coordination (the Cellular Potts result of §7.3; Fig. 5, right). To keep the comparison fair at high core counts we give the synchronous ABC-SMC baseline a *population size equal to the worker count* (a smaller population cannot occupy every core); it then uses all allocated cores and scales steadily, from ≈440 sims/s at 16 workers to ≈6000 at 288 (Table 5). On this free simulator the asynchronous method leads while the job stays within a few nodes (≈4.5× at 16 workers, ≈2× at one full node), but its lead *erodes* with scale and the fair synchronous baseline overtakes it beyond three nodes, where the growing all-to-all coordination cost dominates and the asynchronous method leaves most of the allocation idle. This is the honest boundary of the asynchronous advantage on a cheap, homogeneous simulator; it is largest exactly in the heterogeneous and straggler-prone regimes where barriers otherwise waste compute. The archive size sets where that boundary falls, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$: enlarging the archive tenfold to $k = 1000$ costs throughput at every worker count — 39% at the single-node peak (6240 → 3804 sims/s) — and brings the crossover with the fair baseline in from beyond three nodes to a single node (Table 5). Sweeping k across $\{50, 100, 200, 400, 800, 1000\}$ at every worker count resolves that cost into a smooth monotone decline in throughput and a single step in the crossover, which moves in from beyond three nodes to three nodes at $k \geq 400$ (Table 7). The archive

Table 5 Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 5, left, shows the inter-quartile spread). Asynchronous (ours) at both archive sizes vs. synchronous baseline. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 the synchronous baseline is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous method peaks on one node and then declines as multi-node coordination dominates, so the baseline overtakes it beyond three nodes at $k = 100$. Enlarging the archive tenfold costs throughput at every scale — the per-arrival proposal reconstruction is $\mathcal{O}(k)$ — and moves that crossover in to a single node.

Workers	1	4	16	48	144	192	240	288
Nodes	<1	<1	<1	1	3	4	5	6
Synchronous	105	96	437	2952	3757	4322	4539	6034
Asynchronous ($k = 100$)	149	369	1983	6240	4195	2615	2088	1729
Asynchronous ($k = 1000$)	106	133	620	3804	2804	1957	1587	1336

size is therefore a systems cost as well as a statistical knob — but, put on the same axis as the calibration it buys, the two turn out *not* to trade off over this range: the best-calibrated archive is also within 7% of the fastest, and every archive larger than $k = 100$ is both slower and worse calibrated (§7.2, Table 7).

7.2 Posterior validity (RQ1)

Having established the systems advantage, we ask what it costs in inference quality. Across calibration and matched-budget posterior recovery the reported streaming posterior — the full-history AMIS estimator (6) — is comparable to the matched synchronous baseline in low dimensions and under multimodality, and in four dimensions errs toward conservatism (it mildly over-covers) rather than over-confidence.

Calibration.

Table 8 reports SBC empirical coverage on the Gaussian-mean model (1000 trials, matched simulation budget), for the reported full-history AMIS posterior. The asynchronous method tracks the nominal level closely (0.51/0.80/0.90/0.94 at the 0.50/0.80/0.90/0.95 levels), within Monte Carlo error of nominal at every level; the synchronous baseline under-covers at every level (0.35/0.63/0.75/0.83), i.e. it is overconfident. At 1000 trials a coverage estimate carries a Monte Carlo standard error of only $\approx \pm 0.007$–0.016, well below the $\approx$0.12–0.17 gap between the methods, so the ordering is resolvable rather than noise: on this benchmark the streaming estimator is *better* calibrated than the matched synchronous baseline. Reporting the estimator over the full evaluated history rather than the top-k archive alone matters even here: in a paired 500-trial run that changes only the reported support, the truncated archive mildly under-covers at the top levels (0.89/0.92 at the 0.90/0.95 levels), which the full-history report corrects to nominal — a reporting-support effect that grows with dimension, as the multimodal and four-dimensional tests below make explicit. This test is on a one-dimensional model with an analytic posterior; the higher-dimensional

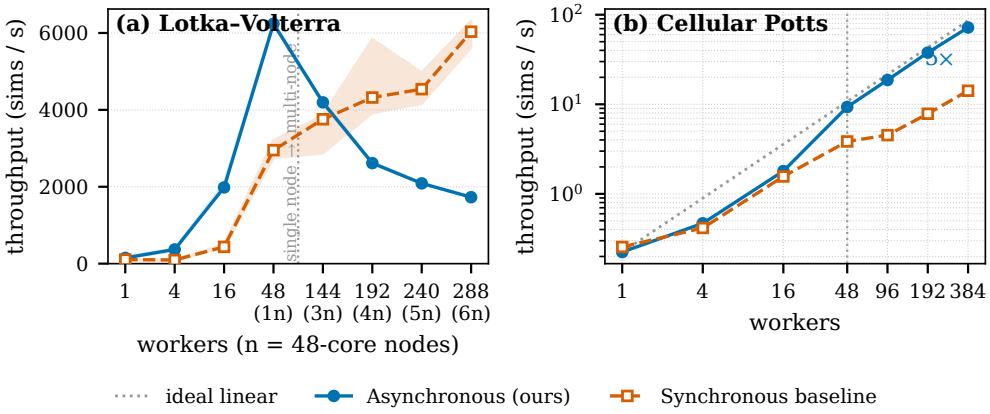

Fig. 5 Strong scaling: median simulation throughput versus worker count at a fixed wall-clock budget, asynchronous (ours) vs. a synchronous baseline given *population size equal to the worker count* so it can occupy every core. **Left:** Lotka–Volterra (near-instant simulator, ≈ 2 ms/eval; 180 s budget; categorical x-axis; bands are inter-quartile ranges over five replicates; node count n in labels). With no per-evaluation compute to distribute, the asynchronous method's single-population all-to-all coordination cost grows with the worker count: throughput rises within one node, peaks there, and then declines across multi-node jobs, so its lead over the fair, steadily scaling baseline erodes and the baseline overtakes it beyond three nodes. **Right:** Cellular Potts (costly `cellsInSilico` simulator; 1800 s budget; log–log; medians over replicates). Real per-evaluation cost amortizes the coordination, so the asynchronous method tracks ideal-linear scaling (dotted) to eight nodes (384 workers, ≈97% parallel efficiency) while the barrier-bound baseline scales only sub-linearly, a ≈5× gap at 384 workers that *grows* with scale.

and multimodal settings where the archive/proposal approximations are most likely to bite are probed separately below. Figure 7 shows the corresponding coverage curve and rank histogram.

Calibration under multimodality and in higher dimensions.

To probe exactly those settings, we run SBC (1000 trials, matched simulation budget) on two harder targets, reporting the same full-history AMIS posterior. The first is a *multimodal* model $y_i \sim \mathcal{N}(\theta^2, \sigma^2)$: the sign ambiguity gives a symmetric bimodal posterior at $\pm\sqrt{\bar{y}}$, the case a top-k archive is most at risk of collapsing onto a single mode. It does not collapse — the archive retains *both* modes in 85.7% of trials and the mode containing the true value in 86.7% (the remainder are largely near-θ=0 trials where the two modes merge), the importance weights stay near-uniform (median effective-sample-size fraction 0.99, median maximum normalised weight 0.011), and coverage tracks the nominal level (0.50/0.82/0.91/0.96), calibrated within Monte Carlo error. The second is the *four-dimensional* g-and-k model, where the reporting-support choice bites. The reported full-history posterior is *conservative*: it mildly over-covers at every level and parameter (e.g. empirical coverage 0.62/0.91/0.96/0.99 at the 0.50/0.80/0.90/0.95 levels for the location parameter A, and similarly for B, g, k), because the finite-ϵ smooth-ABC posterior it reports is over-dispersed at the loose bandwidth a four-dimensional budget reaches. The efficient top-k archive used for

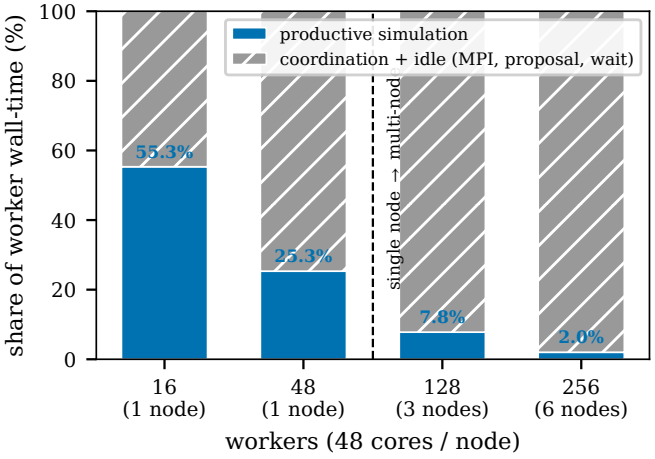

Fig. 6 Measured decomposition of asynchronous worker wall-time on Lotka–Volterra ($k = 100$; 180 s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated) — the share of worker wall-clock spent inside the simulator — collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate exactly as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 5 is therefore a measured property of coordinating a near-instantaneous simulator, not an inferred one.

online proposal sits at the opposite end: it is over-concentrated and would instead *under*-cover (0.85/0.89 at the 0.90/0.95 levels for A), with healthy weights throughout (median effective-sample-size fraction 0.81), so the miscalibration is a reporting-support effect, not weight degeneracy. The calibrated posterior is therefore *bracketed* between the over-concentrated archive core and the over-dispersed full history, and the amount of history reported is the dispersion knob: widening the reported support from the archive toward the full history raises coverage monotonically and continuously through nominal — for A, 0.85/0.89 → 0.93/0.96 → 0.95/0.98 → 0.97/0.99 at the 0.90/0.95 levels for reported sizes $M = 200/500/1500/\text{full}$, crossing nominal near $M \approx 400$–800 (the shape parameters B and k are already near-nominal at M=500, e.g. 0.89/0.94 for B). This is a property of *how much of the history is reported*, not of the AMIS weighting (whose importance weights stay well-conditioned) or of the budget: doubling the simulation budget to 80,000 per trial leaves the archive coverage essentially unchanged (0.85/0.89 at the 0.90/0.95 levels for A, within Monte Carlo error for B, g, k). We accordingly report the streaming estimator, evaluated over the full evaluated history, as well calibrated in one dimension and under multimodality, and conservative — over-covering rather than over-confident — in four dimensions; an intermediate reported support ($M \approx 400$–800), a larger archive, or the defensive-mixture floor (§9) tighten the four-dimensional report toward exactly nominal.

How calibration depends on the archive size and the snapshot buffer.

The tests above fix k and S per benchmark, which leaves open whether the two efficiency knobs must be retuned as the dimension grows. To settle this we sweep them

directly on a d-dimensional Gaussian-mean target, $y_i \sim \mathcal{N}(\theta, \sigma^2 I_d)$ with $\theta \in \mathbb{R}^d$ under a flat box prior, whose posterior is known exactly at every d (SBC against that exact posterior returns 0.496/0.794/0.905/0.952, confirming the target itself is calibrated, so any departure below is a property of the method). The grid is $d \in \{1, 2, 4, 8, 16\} \times k \in \{50, 100, 200, 400, 800\} \times S \in \{0, 5, 20, 50\}$, 1000 trials per cell, reporting the same full-history estimator (Table 6).

The two knobs behave quite differently. The snapshot buffer S has a threshold and then saturates: at $S = 0$ the estimator is over-confident in low dimensions (mean deviation -0.088 at $d = 1, 2$), $S = 5$ removes essentially all of it (-0.001), and $S = 20$ and $S = 50$ add nothing further — at any dimension. The value of a richer balance-heuristic denominator therefore does *not* grow with d over this range, and the fixed $S = 20$ we use throughout sits comfortably above the knee. The archive size k is the opposite: its effect is large, strongly dimension-dependent, and *not* monotone. Small archives can be severely over-confident in intermediate dimensions — at $d = 8, k = 50$ empirical coverage is 0.29/0.52/0.63/0.71 against the nominal 0.50/0.80/0.90/0.95, a failure that persists at every S — while intermediate archives over-cover in the same regime ($+0.18$ to $+0.20$ at $k = 200$–400). We note that this does not follow the floor a naive count of covariance parameters would predict: the perturbation covariance is a full $d \times d$ matrix estimated from the archive, suggesting $k \gtrsim d(d+1)/2$, yet $d = 8$ fails at $k = 50 > 36$ while $d = 16$ remains within 0.06 of nominal at $k = 50 < 136$. The reading this supports is that $d = 16$ is protected by its own difficulty: with 20,000 simulations over a $[-5, 5]^{16}$ prior the sampler barely concentrates, so the reported posterior stays broad and covers almost by default, whereas $d = 8$ has enough signal to concentrate but not enough to concentrate correctly. Two further runs test that reading rather than leaving it as narrative. First, it predicts that a *higher* dimension should look benign again: at $d = 32$ the coverage is 0.55–0.57, 0.83–0.84, 0.92, 0.96 at the four levels for every $k \in \{50, 100, 800\}$ — mildly conservative and nowhere near the $d = 8$ failure, even at $k = 50$. Second, the failure is not a budget shortfall: doubling the simulation budget at $d = 8, k = 50$ makes coverage slightly *worse* (0.26/0.47/0.58/0.67 against 0.29/0.52/0.63/0.71 at the original budget), exactly as expected if the extra simulations sharpen an already mislocated posterior rather than repairing it. We therefore do not offer a closed-form rule for k; the practical guidance is that the archive size is the dispersion knob (§7.2), that its safe range is problem- and dimension-specific, and that it should be chosen by checking calibration directly rather than by a formula.

The archive size does not, in this range, trade calibration against throughput.

Because k has both a statistical effect (above) and a systems cost — per-arrival proposal reconstruction is $\mathcal{O}(k)$, so a larger archive is slower at every worker count (§7.1) — it is natural to expect the two to pull against each other, and to have to buy calibration with throughput. Measuring both on one axis shows that expectation is wrong over the range we can test. Table 7 puts the throughput cost of k next to its worst-case miscalibration over the dimensions swept, using a throughput grid matched on everything but k (Lotka–Volterra, 180 s wall cap, five replicates, worker counts 1–288:

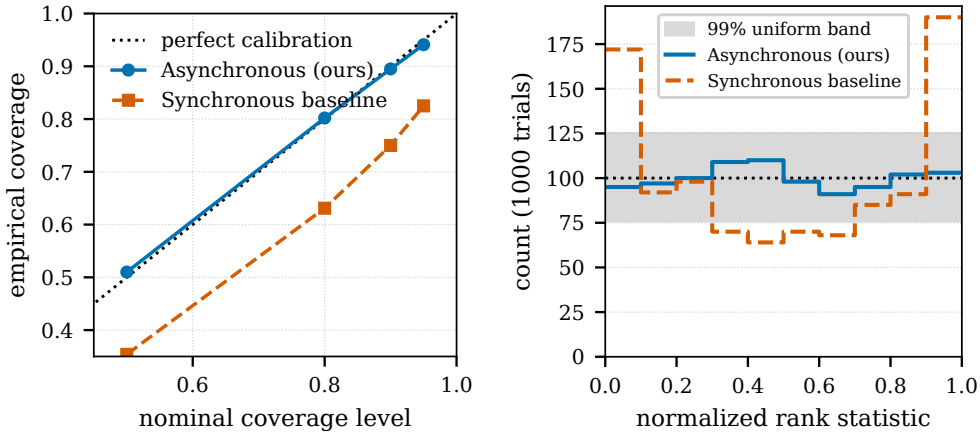

Fig. 7 Simulation-based calibration on Gaussian-mean inference (1000 trials; the reported full-history AMIS posterior): empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.

$k \in \{100, 1000\}$ from the main scaling run and $k \in \{50, 200, 400, 800\}$ from a dedicated k-frontier run sharing that configuration). Of the six archive sizes, only two are Pareto-optimal, and they are the two smallest: $k = 100$ is simultaneously the best-calibrated setting tested (worst deviation 0.064, at $d = 4$) and within 7% of the fastest (6240 vs 6684 simulations/s at the single-node peak). Every larger archive is dominated by it on *both* axes at once — $k = 200$ is 5% slower and nearly three times worse calibrated (0.180), $k = 400$ is 14% slower and 0.195, and $k = 800$ costs 31% of throughput to end up still slightly worse calibrated than $k = 100$ (0.074). The only genuinely faster archive, $k = 50$, is the catastrophic one (0.250 at $d = 8$). So the practical shape of the k decision is not a frontier to negotiate but a floor to clear: go far enough above the small-archive failure region, and then stop, because further enlargement buys no calibration and costs throughput monotonically.

Two qualifications keep this from being read too broadly. The join is on k alone and the two axes come from different benchmarks by design — calibration needs a target whose posterior is known at every d, throughput needs the regime where an $\mathcal{O}(k)$ per-arrival cost is most exposed. That second choice makes the throughput column an *upper* bound on the relative cost of k: Lotka–Volterra evaluations take $\approx$2 ms, so coordination dominates there, and on a cost-bearing simulator the per-evaluation work amortizes the archive operation (§7.3). The conclusion is therefore conservative in the right direction — even where k is most expensive, the best-calibrated archive is nearly free. What k *does* change at scale is where the fair synchronous baseline overtakes the asynchronous method, and there it acts as a step rather than a smooth trade: the crossover sits beyond three fully packed nodes for $k \leq 200$ and moves in to three nodes for $k \geq 400$ (last column).

Posterior recovery.

On g-and-k the asynchronous method reaches a slightly *lower* Wasserstein distance to the truth than the synchronous baseline at matched wall-clock budget (0.078 vs.

Table 6 Calibration sensitivity to the archive size k and the AMIS snapshot buffer S on the d-dimensional Gaussian-mean target (1000 trials per cell; entries are the mean signed deviation of empirical from nominal coverage over the 0.50/0.80/0.90/0.95 levels and the d parameters). Negative is over-confident, positive is conservative; 0.000 is exactly calibrated. **Left:** varying k at $S = 20$. **Right:** varying S at $k = 100$.

	archive size k (at $S = 20$)				
	50	100	200	400	800
$d = 1$	-0.014	-0.003	+0.006	+0.006	+0.074
$d = 2$	-0.020	-0.006	+0.009	+0.030	+0.072
$d = 4$	+0.025	+0.064	+0.094	+0.133	+0.063
$d = 8$	**-0.250**	+0.055	+0.180	+0.195	+0.061
$d = 16$	-0.047	+0.028	+0.049	+0.053	+0.055

	snapshot buffer S (at $k = 100$)			
	0	5	20	50
$d = 1$	**-0.088**	-0.001	-0.003	-0.005
$d = 2$	**-0.088**	-0.001	-0.006	-0.003
$d = 4$	+0.039	+0.068	+0.064	+0.064
$d = 8$	+0.065	+0.006	+0.055	+0.074
$d = 16$	-0.010	+0.010	+0.028	+0.034

0.090, inter-quartile ranges $[0.077, 0.083]$ and $[0.082, 0.093]$), and on Lotka–Volterra the two are comparable with the asynchronous method marginally lower (0.316 vs. 0.332, overlapping inter-quartile ranges). On the expensive Cellular Potts simulator the ordering reverses and the asynchronous method is *modestly but consistently behind*: the async−sync difference is positive across 87.5% of the budget, ending at +0.043 with an inter-quartile envelope of $[0.019, 0.052]$ that excludes zero, and it never exceeds ≈0.06 against a Wasserstein scale of ≈0.4 (Figs. 8c and 9). One property of this benchmark bounds how that gap should be read: the Wasserstein-to-reference *rises* with compute for both methods (asynchronous $0.37 \to 0.48$), because the Cellular Potts parameters are only weakly identified (§7.3) and the comparison is against a reference point rather than a known posterior, so a diffuse cloud scores better than a posterior that has concentrated slightly off that reference. The metric therefore penalizes concentration on this target, and the asynchronous method — which completes ≈2× the evaluations in the same budget — concentrates fastest. Per-parameter posteriors and joint marginals are consistent with the known/reference values. In the straggler runs (which compute posterior quality) the final Wasserstein distances are comparable between methods (asynchronous ≈ 0.07–0.08 vs. synchronous ≈ 0.07). On the Gaussian-mean benchmark, which has a closed-form posterior, the asynchronous method recovers the posterior mean to within ≈ 0.002–0.038 absolute error (comparable to the synchronous baseline, ≈ 0.006–0.021, and far tighter than a rejection-ABC reference) and tracks the

Table 7 The archive size on both of its axes at once. **Throughput**: median asynchronous throughput at the single-node peak (48 workers) on Lotka–Volterra, 180 s wall cap, five replicates ($k \in \{100, 1000\}$ from the main scaling run, $k \in \{50, 200, 400, 800\}$ from the matched k-frontier run), and its change relative to the k=100 we use throughout. **Calibration**: worst absolute deviation of empirical from nominal coverage over the dimensions of the d-dimensional Gaussian-mean sweep at S=20 (Table 6), with the dimension attaining it. **Crossover**: fully packed nodes at which the fair synchronous baseline (population = worker count) overtakes the asynchronous arm. Only k=50 and k=100 are Pareto-optimal; every larger archive is both slower *and* worse calibrated than k=100. The sweep did not include k=1000.

k	sims/s at W=48	vs. k=100	worst $\lvert\Delta\text{cov}\rvert$	at	baseline overtakes
50	6684	+7%	**0.250**	d=8	beyond 3 nodes
100	6240	—	**0.064**	d=4	beyond 3 nodes
200	5904	−5%	0.180	d=8	beyond 3 nodes
400	5393	−14%	0.195	d=8	3 nodes
800	4331	−31%	0.074	d=1	3 nodes
1000	3804	−39%	—	—	3 nodes

Table 8 SBC empirical coverage on Gaussian-mean inference (parameter μ, 1000 trials; Monte Carlo standard error $\approx \pm 0.007$–0.016), for the reported full-history AMIS posterior. Closer to the nominal level is better.

Method	0.50	0.80	0.90	0.95
Synchronous baseline	0.35	0.63	0.75	0.83
Asynchronous (ours)	0.51	0.80	0.90	0.94

baseline's Wasserstein-vs-time trajectory throughout the budget (Fig. 10); the SBC calibration above is unaffected.

7.3 Realistic simulator: Cellular Potts (RQ3)

The Cellular Potts benchmark runs end-to-end with the real `cellsInSilico` simulator, a realistic, expensive, heterogeneous-runtime workload and the regime the method targets.

Strong scaling.

Because each Cellular Potts evaluation carries real cost, the per-arrival coordination that capped Lotka–Volterra is amortized, and the asynchronous method scales almost linearly all the way to eight nodes (Fig. 5, right): median throughput rises from 0.2 to 72.0 simulations/s over 1 to 384 workers and, unlike the near-instantaneous Lotka–Volterra case, keeps scaling *across* the single-node→multi-node boundary, each node-doubling roughly doubling throughput (9.3 → 18.7 → 37.8 → 72.0 sims/s at 48 →

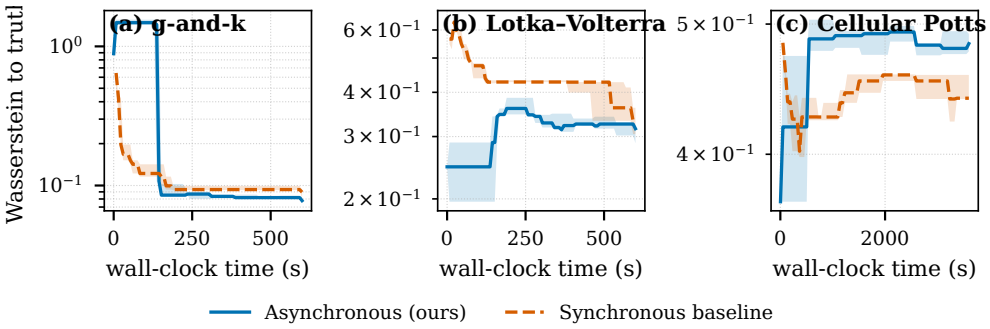

Fig. 8 Posterior quality (Wasserstein distance to the truth/reference, lower is better) versus wall-clock time on g-and-k (left), Lotka–Volterra (middle), and Cellular Potts (right), asynchronous vs. synchronous; shaded bands are inter-quartile ranges over five replicates. On g-and-k the asynchronous method reaches a slightly lower Wasserstein distance within the budget, on Lotka–Volterra the two are comparable, and on Cellular Potts the asynchronous method is modestly but consistently behind. Note that on the weakly identified Cellular Potts target the distance-to-reference *increases* with compute for both methods, so the metric penalizes the method that concentrates fastest (§7.2).

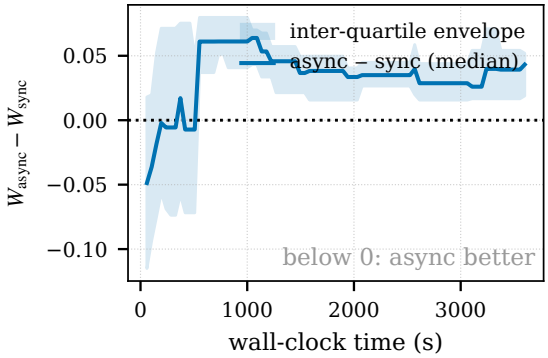

Fig. 9 Cellular Potts posterior-quality *difference* (asynchronous minus synchronous Wasserstein to the reference) versus wall-clock time, with a zero reference line and an inter-quartile envelope over five replicates (the per-method Cellular Potts curves of Fig. 8c, differenced). The difference is positive — asynchronous behind — across 87.5% of the budget, ending at +0.043 with an envelope of $[0.019, 0.052]$ that excludes zero; it straddles zero only over the first ≈510 s. The gap is nonetheless small in absolute terms, never exceeding ≈0.06 against an absolute Wasserstein of ≈0.4, and it is measured against a reference point on a weakly identified target whose distance-to-reference grows as either method concentrates (§7.2, §7.3).

96 → 192 → 384 workers), for ≈ 97% parallel efficiency relative to the single-node rate. The synchronous baseline, given a population equal to the worker count so it too can occupy every core, scales only *sub-linearly* over the same range (3.8 → 4.5 → 7.8 → 14.2 sims/s at 48 → 96 → 192 → 384 workers, ≈50% efficiency), so at 384 workers the asynchronous method sustains ≈5× its throughput, a gap that *widens* with scale. This is the regime the method is built for: an expensive, heterogeneous simulator where the generation barrier, not per-arrival overhead, is the binding cost. Worker utilization makes the mechanism explicit (Fig. 11): the asynchronous method

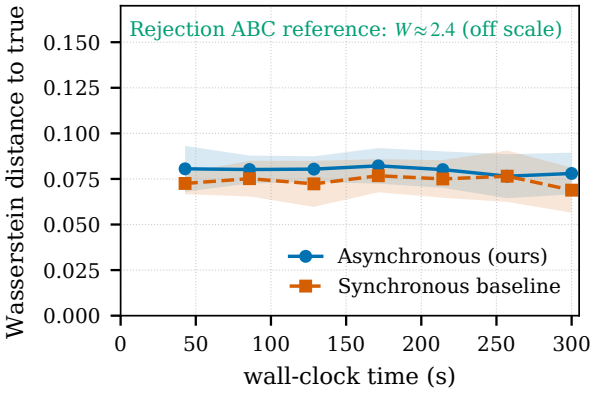

Fig. 10 Gaussian-mean posterior recovery: Wasserstein distance to the true μ versus wall-clock time, with replicate confidence bands. The asynchronous method (archive) tracks the matched synchronous baseline (generation) throughout the budget, both far below the annotated rejection-ABC reference ($W \approx 2.4$).

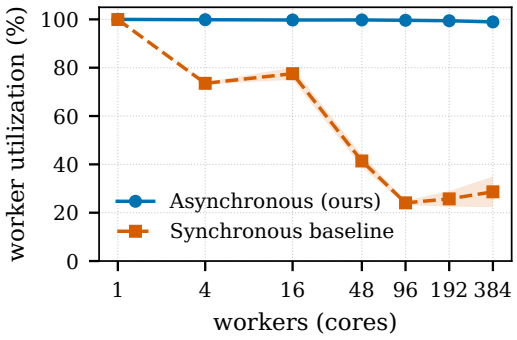

Fig. 11 Cellular Potts worker utilization (fraction of worker wall-clock spent inside the simulator) versus worker count (1800 s budget; error bars are ±1 s.d.); the synchronous baseline uses a population equal to the worker count (Fig. 5, right). The asynchronous method stays ≈98.9–99.8% utilized at every scale, while the synchronous baseline, even filling every core within a generation, then idles at the barrier waiting for the slowest simulation, so its utilization falls to ≈24–29% (41% → 24% → 26% → 29% at 48/96/192/384 workers, i.e. ≈70–76% idle at every multi-node scale). This barrier idle is the measured mechanism behind the synchronous method's sub-linear scaling in Fig. 5 (right).

keeps workers ≈98.9–99.8% busy across all scales, whereas the synchronous baseline, even with every core filled within a generation, then waits for the slowest simulation at the barrier, so its utilization falls to ≈24–29% (41% → 24% → 26% → 29% from 48 to 384 workers, i.e. ≈70–76% idle at every multi-node scale). This barrier idle, not per-arrival overhead, is what holds the synchronous throughput to sub-linear scaling while the asynchronous method tracks the ideal. (We report medians throughout.)

The barrierized twin at this scale.

The twin comparison of §7.1 isolates synchronization on the two synthetic benchmarks; here we run it on the real simulator, at the same four worker counts and the same

k=100, simulation-limited at 40 evaluations per worker (Table 9). Two things come out of it, and the second is the more interesting.

First, the barrier alone costs 1.9× at one and two nodes and 4.4× at eight — a gap that widens with scale, and that lands close to the 2.4–5.1× we measure against the external pyABC baseline over the same range. That correspondence is what the twin was built to check: on this workload the barrier accounts for essentially the whole cross-algorithm gap, so the comparison against pyABC is not inflated by the other differences between the two implementations. It is also the opposite regime to the straggler benchmark, where the twin ran far *slower* than pyABC because pyABC's latecomer discarding rescues it from waiting on a permanently slow rank; with no straggler and a cost-bearing simulator, the two synchronous schedules cost about the same.

Second, the barrier does not merely reduce throughput — it makes throughput *unpredictable*. The asynchronous arm's five replicates agree to within 1.3% at every scale (coefficient of variation 0.6–1.3%), while the twin's spread across the same seeds is 27–75%: at 48 workers its replicates run at 1.7, 2.1, 4.9, 6.0 and 5.9 simulations/s. The mechanism is the one the utilization measurement already shows: a generation ends only when its slowest evaluation does, so the whole job inherits the cost of the most expensive parameter draw in each generation, and on this benchmark simulation cost varies strongly with the parameters. Asynchrony removes that coupling — a slow draw delays only its own worker — which is why the asynchronous arm's throughput is reproducible to within a percent while its barrierized twin, running the identical proposals on identical seeds, varies by a factor of three.

Two honesty notes on this table. The twin becomes an unreliable instrument at high rank counts: it intermittently deadlocks at $\geq$ 192 ranks — once in the MPI teardown, once after two evaluations — and three of six replicate attempts at 384 ranks failed to complete, leaving three and four replicates at 384 and 192 workers against five elsewhere. And the hangs correlate with slowness (the two replicates that hung repeatedly were the slowest to make progress when they did run), so dropping them biases the twin's throughput *upward* and the reported ratios are, if anything, conservative.

Posterior quality.

On the inference side the asynchronous method produces marginals of similar shape to the synchronous baseline (Fig. 12), though it trails modestly on distance-to-reference (§7.2): the cell-motility parameter concentrates near its reference value, while the division rate is only weakly identified (for *all* methods, the baseline included). This weak identification is a structural property of the inference problem, not of asynchrony: at this scale the division-rate and motility parameters are near-degenerate (both chiefly modulate the cell count that the summaries resolve), so neither method separates them and both improve only modestly on rejection ABC. The matched comparison therefore isolates the systems advantage as largely *orthogonal* to identifiability: both methods are limited chiefly by the problem. On the summary metric the asynchronous method is modestly behind over the full budget (+0.043 in Wasserstein-to-reference against a scale of ≈0.4; §7.2), so we report its throughput advantage as bought at a small

Table 9 Barrierized twin on the Cellular Potts workload: median throughput (simulations/s) over the available replicates, k=100, simulation-limited at 40 evaluations per worker. CV is the coefficient of variation across replicates. The barrier's cost widens with scale (1.9 → 4.4×) and closely tracks the 2.4–5.1× measured against the external pyABC baseline over the same range, so on this workload the barrier accounts for essentially the whole cross-algorithm gap. The twin's replicate spread (27–75%) against the asynchronous arm's 0.6–1.3% is the second finding: a generation inherits the cost of its most expensive parameter draw, so synchronization makes throughput unpredictable as well as lower. The twin intermittently deadlocks at $\geq$ 192 ranks, which is why 192 and 384 workers have four and three replicates; the hangs correlate with slow progress, so excluding them if anything understates the barrier's cost.

Workers (= cores)	48 (1 node)	96 (2)	192 (4)	384 (8)
Asynchronous (ours)	9.33	18.67	37.75	71.98
CV, n=5	0.9%	1.3%	0.6%	1.3%
Barrierized twin	4.87	10.00	17.79	16.52
CV	50.7%	53.8%	26.9%	75.1%
replicates	5	5	4	3
Ratio, twin	1.9×	1.9×	2.1×	4.4×
Ratio, pyABC baseline	2.4×	4.2×	4.8×	5.1×

quality cost on this benchmark rather than at none — with the caveat that the metric rewards dispersion on a weakly identified target, and that the asynchronous method, concentrating fastest, is the one it penalizes most. Establishing the same quality with a synchronous matched baseline at the still larger scales the method targets would itself be prohibitively wasteful, precisely the cost the barrier-free design removes.

7.4 Ablation

Reverting the smooth kernel to a hard indicator slightly raises the Wasserstein-to-truth relative to the full method (though within its confidence interval), while removing the AMIS cumulative-mixture reweighting is approximately neutral on this uniform-runtime analytic target (Fig. 13). On the Gaussian-mean benchmark the full method attains a final Wasserstein-to-truth of 0.073 (CI $[0.067, 0.079]$); the hard-indicator kernel raises it to 0.076 (inside that interval), whereas dropping AMIS leaves it essentially unchanged (0.072; Fig. 13a), with the remaining hyperparameter variants scattered around the full-method reference and degrading only toward the largest perturbation scales (0.084–0.087). That AMIS is neutral here is consistent with the coupling study (§7.1), where removing AMIS likewise left the posterior essentially unchanged: with no runtime–parameter coupling on this uniform-runtime target, there is no over-representation for the reweighting to reshape. The AMIS-isolation trajectory (Fig. 13b) accordingly shows the full method and the no-AMIS variant tracking each other throughout the converged regime (both $\approx$ 0.072–0.073 after 40 s).

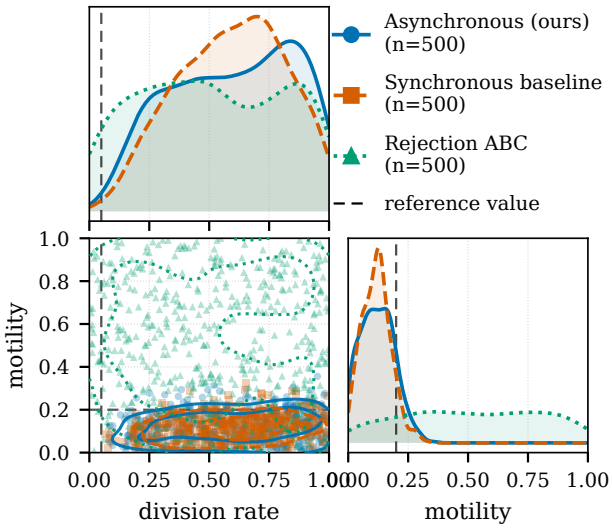

Fig. 12 Cellular Potts posterior (division rate, motility) on the real `cellsInSilico` simulator: asynchronous (blue), synchronous baseline (red), and rejection ABC (green), with dashed lines at the reference values. The asynchronous posterior is drawn from its AMIS estimator (500 resampled draws); the synchronous baseline and rejection ABC show their final populations (500 each). The asynchronous method and the synchronous baseline both place their motility mass at low values while the division rate is only weakly identified by the summaries for all methods, and both concentrate more tightly than rejection ABC.

7.5 Sensitivity

Across the full hyperparameter grid *on the Gaussian-mean benchmark* (the analytic-posterior sanity model) the asynchronous method's posterior quality is stable, with no region of catastrophic failure (Fig. 14). The Wasserstein distance to the true parameter stays within ≈ 0.08–0.15 across the grid (each cell averaged over five replicates and the two smooth kernels); the archive size k matters only marginally *for point quality at this dimension* (its effect on calibration in higher dimensions is a separate and much larger story, Table 6), and quality degrades only mildly toward the largest perturbation scales and initial tolerances. The tolerance scheduler is inert under a smooth kernel — the asynchronous selector chooses ϵ by ESS-retention bisection regardless of the scheduler type — so we fixed it to the acceptance-rate rule rather than sweeping a variable that cannot change the smooth-kernel result. We therefore did not find delicate tuning necessary to reach the quality reported above *on this benchmark*. We did not repeat the full grid on the costlier non-Gaussian benchmarks, so this robustness statement is scoped to the analytic-posterior model and to the qualitative trends observed there (scheduler-insensitive; mild, monotone sensitivity to perturbation scale and initial tolerance) rather than asserted as a general guarantee. Two further scope limits matter. This grid measures *point quality* (Wasserstein to the truth) in *one* dimension; it says nothing about calibration, and the archive size that looks inconsequential here is decisive for coverage in higher dimensions — the dimension-resolved sweep of §7.2 (Table 6) finds a genuinely catastrophic region ($d = 8$, $k = 50$: coverage 0.29 at the

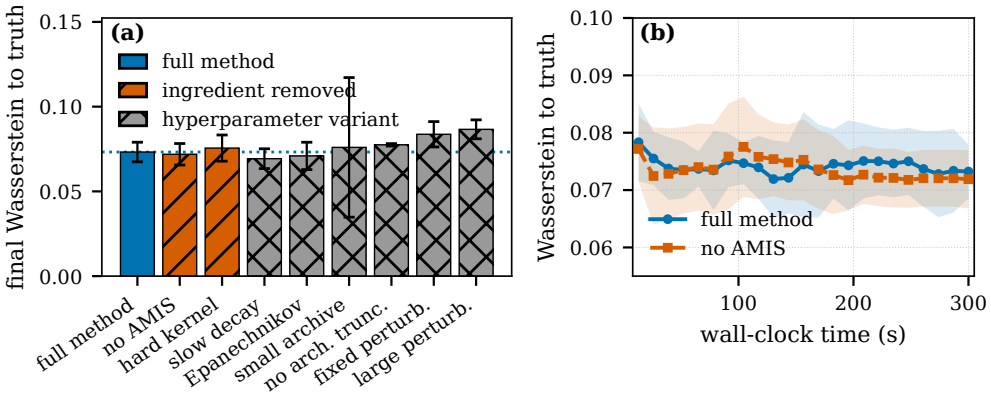

Fig. 13 Ablation on the Gaussian-mean benchmark (analytic posterior; 48 workers, 300 s budget, $k = 100$, five replicates, asynchronous method). (a) Final Wasserstein-to-truth for the full method (blue, solid), the two ingredient removals (vermillion, hatched: no AMIS, hard kernel), and hyperparameter variants (grey, cross-hatched); colour and hatch encode the class redundantly, error bars are 95% CIs, and the dotted line marks the full-method mean. (b) AMIS isolation: full method vs. no-AMIS on a shared wall-clock grid (bands are ± 1 s.d. over replicates), zoomed to the converged regime; on this uniform-runtime analytic target the two variants track each other.

0.50 level) that a one-dimensional quality grid cannot see. "No failure region" should therefore be read as a statement about one-dimensional point quality only.

8 Discussion

The results support the three claims. Asynchronous execution helps most where it should: under a persistent straggler and under runtime heterogeneity, the generation barrier is the dominant cost and removing it preserves throughput that the synchronous baseline loses. For homogeneous, near-instantaneous simulators at large worker counts the asynchronous coordination overhead does not pay off: the multi-node Lotka–Volterra points, where the fair synchronous baseline overtakes the asynchronous method, make this honest boundary explicit. On the realistic Cellular Potts workload (the cost-bearing regime the design targets) the method scales almost linearly while the synchronous baseline, held at the generation barrier even when given a matching population, scales only sub-linearly — at a small measured cost in posterior quality on that benchmark (§7.2), on a metric that penalizes the concentration the extra throughput buys.

9 Limitations

We are explicit about what §4 does and does not give. (i) AMIS estimators are consistent but not finite-sample unbiased, since proposals depend on past particles; a local-decision rule in the spirit of [19] that recovers finite-sample unbiasedness for ABC is future work. (ii) The bandwidth floor $\epsilon_\infty > 0$ in Condition 3 means the theorem

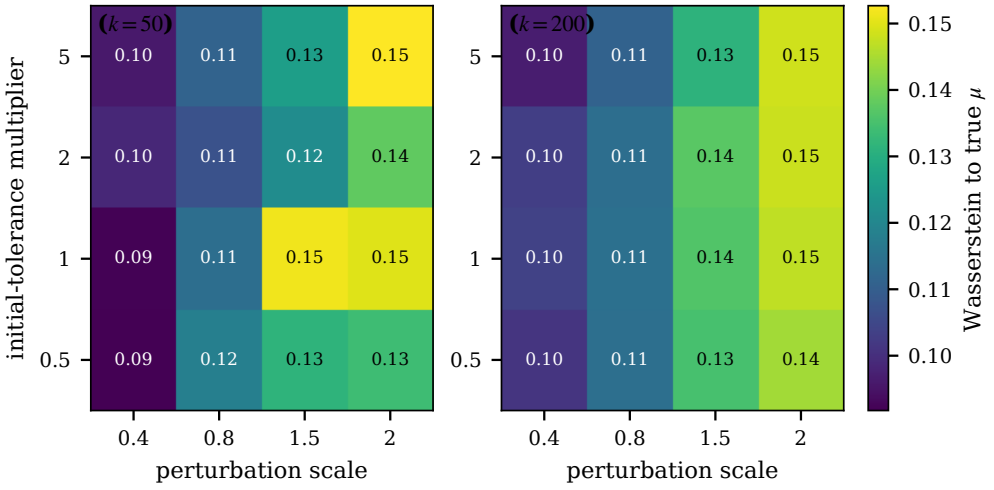

Fig. 14 Hyperparameter sensitivity of the asynchronous method, measured by posterior quality (Wasserstein distance to the true μ, lower is better; averaged over replicates and smoothing kernel). Each panel sweeps perturbation scale (x) against the initial-tolerance multiplier (y); panels are faceted by archive size k. Quality is stable across the grid (Wasserstein $\approx$ 0.08–0.15) with no failure region; the tolerance scheduler is inert under the smooth kernel and is held fixed, while larger perturbation scales degrade quality only mildly.

characterizes the smooth-ABC posterior at the limit bandwidth, not the exact posterior at $\epsilon = 0$. (iii) Condition 1 is assumed for the specific archive-evolution rule, not proved. (iv) Keeping every accepted particle (unlike pyABC's discard-latecomers rule) over-represents fast-simulator parameter regions in the raw archive. The parameter-coupled-runtime study (§7.1, Fig. 4) quantifies this: steepening the coupling cuts throughput as the fast region comes to dominate the raw counts, yet — across the sweep, a steeper regime, and with or without AMIS — neither the unweighted posterior mean nor the AMIS-reweighted posterior's mean drifts with the coupling (AMIS reshapes the tails, not the mean), and a drain-after-deadline test finds the completion-time censoring negligible in effect at these levels (numbers there). We therefore do not correct for it algorithmically and do not claim the AMIS reweighting cancels it: dividing by the proposal density corrects proposal adaptation, not selection by completion time, so a formal correction would need completion probabilities or an anytime-sampling argument, and a residual finite-sample effect at stronger couplings cannot be ruled out — both left to future work. (v) The snapshot buffer is fixed at $S = 20$; adaptive sizing is a natural extension. (vi) Simulation-based calibration of the reported posterior — the full-history AMIS estimator (6) — shows it well calibrated in one dimension and robust to multimodality (retaining both modes of a bimodal target in ≈86% of trials), and conservative rather than over-confident on the four-dimensional g-and-k model, where it mildly *over*-covers. This four-dimensional behaviour is a reporting-support effect, not a weighting or budget failure (§7.2): the efficient top-k archive used for online proposal is over-concentrated and would under-cover, the full evaluated history is over-dispersed and over-covers, and coverage rises

monotonically through nominal as the reported support grows from one to the other, so the calibrated posterior is bracketed between the two. We report the conservative full-history end by default; an intermediate reported support, a larger archive, or the persistent defensive-mixture component — which the prior floor already approximates — would tighten the four-dimensional report to nominal. (vii) The synchronous baseline is matched on the ABC kernel and bandwidth schedule, but it still differs from our method in more than the barrier: it adds a probabilistic-rejection step, its generation length depends on the resulting acceptance probability, and it retains classical population weighting rather than the streaming estimator. Matching the kernel removes the acceptance *rule* as a confound, and the *barrierized twin* of §7.1 (Table 3) closes the remaining gap on the straggler benchmark: the same propagator with a collective barrier before each breed runs 21–402× slower than the asynchronous arm at unchanged posterior quality, so on that workload the gain is attributable to synchronization rather than merely dominated by it. The granularity objection — that the twin synchronizes every W evaluations, a finer generation than the baseline's population of 100 — is measured rather than assumed away: re-running the twin with the barrier coarsened to the baseline's population changes its throughput by less than 1% at every slowdown from 5× up, and matters (2.3–3.3×) only at the controls where barrier latency and not straggler waiting is the bottleneck (Table 3). The twin has since been repeated under runtime heterogeneity (Table 4), where it costs 1.1× at the σ=0 control and 25.3× at σ=2 at the baseline-matched granularity, so the attribution now covers both synthetic HPC pathologies rather than only the sharpest one. That test also shows the cross-algorithm comparison to be conservative in the direction that matters: pyABC's own penalty grows only to 4.8×, because it discards latecomers instead of waiting for the slowest draw (item iv), so the barrier's unmitigated cost exceeds the gap we report against it. The twin has since also been run at scale on the Cellular Potts workload (Table 9), where it costs 1.9× at one node and 4.4× at eight — close to the 2.4–5.1× measured against pyABC over the same range, so on the realistic simulator the barrier accounts for essentially the whole cross-algorithm gap. The attribution therefore now covers all three benchmarks on which we report systems gains. What remains is a limitation of the twin as an *instrument* rather than of the attribution: it intermittently deadlocks at ≥ 192 ranks, so its Cellular Potts points at 192 and 384 workers rest on four and three replicates rather than five, and because the hangs correlate with slow progress their exclusion biases the twin's throughput upward — making those two ratios conservative rather than optimistic. Separately, on a methodological note for reproducibility: the post-run retroactive estimator builds its denominator from $m \leq S = 20$ history-reconstructed proposals (with a defensive prior floor), not the full evaluated history; for a near-instantaneous simulator whose history reaches many millions of particles the pass is evaluated in bounded-memory chunks, which changes memory, not the estimate.

10 Conclusion

We presented a generation-free, single-arrival-driven ABC algorithm that combines smooth-kernel ABC with streaming AMIS reweighting, is stateless and crash-recoverable, and integrates into Propulate's barrier-free island model. Its *idealized* full-mixture estimator inherits AMIS's asymptotic consistency and central-limit guarantees under explicit conditions; the reported posterior, a bounded-memory approximation of it evaluated over the full evaluated history, is validated empirically by simulation-based calibration — well calibrated in low dimensions and under multimodality, and conservative rather than over-confident in higher dimensions, where we trace the residual miscalibration to how much of the evaluated history is reported (the efficient top-k proposal would under-cover; the full history over-covers) rather than to the weighting. Against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule, the method matches or exceeds the baseline's posterior quality on the analytic and Lotka–Volterra targets and trails it modestly on the weakly identified Cellular Potts target, while delivering substantial HPC benefits on heterogeneous and straggler-prone workloads, including that realistic Cellular Potts simulation. Asynchronous, generation-free ABC is a promising approach for large-scale simulator-based inference on modern HPC systems.

Acknowledgements. *To be added in the final version.*

Declarations

Funding
To be added in the final version.

Conflict of interest/Competing interests
The authors declare no competing interests.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Data availability
The experiment outputs backing every figure and table (raw per-particle records, per-rank phase-timing logs, and the throughput/utilization/coverage summary tables), together with the resolved run configurations, will be deposited in a public repository upon publication.

Materials availability
Not applicable.

Code availability
The full implementation (the stateless Propulate propagator, the matched-kernel pyABC baseline, and the analysis and plotting scripts that regenerate every figure and table from the deposited outputs) will be released in a public repository upon publication.

Author contribution
To be added in the final version.

Appendix A Implementation Details

Tolerance schedulers.

Every scheduler is a pure function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum ϵ_{hist} (§3.2), so the effective tolerance can only decrease. Let m denote the extra-individual margin (default $m = k$). The three base rules act on the individuals *within* the current bandwidth, i.e. the hard-threshold set $\{i : \rho_i < \epsilon\}$. (This bandwidth-membership count is used *only* to drive the tolerance schedule; it is distinct from the smooth-kernel weight $K_\epsilon(\rho_i)$ that defines the proposal mixture and the reported posterior, for which every evaluated particle contributes continuously. We avoid the word "accepted" here precisely because acceptance is probabilistic under a smooth kernel.):

- **Quantile.** $\epsilon \leftarrow$ the lower-rank (non-interpolated) p-th percentile of the accepted losses, default $p = 50$; applied only once at least $k + m$ individuals are accepted.
- **Geometric decay.** $\epsilon \leftarrow \epsilon_0 \, \gamma^{\,e}$ after each completed epoch e of $k + m$ accepted individuals, default $\gamma = 0.9$.
- **Acceptance rate.** Over a sliding window of the most recent $k + m$ individuals, if the acceptance rate exceeds r_{hi} (default 0.3) tighten by a factor 0.9, otherwise hold; the rule never expands ϵ (the low-rate guard $r_{\text{lo}} = 0.1$ simply holds).

With a smooth (Gaussian or Epanechnikov) kernel the schedulers run in a *kernel-aware* mode: instead of the hard $\rho < \epsilon$ count, ϵ is chosen by bisection so that the per-step kernel-weighted effective-sample-size retention ratio meets a target (default 0.95), which keeps the bandwidth schedule comparable across kernel choices.

Proposal covariance.

The archive A_n is the top-k history members by smallest discrepancy. Component weights are formed in log-space as $\tilde{W}_j \propto w_j \, K_{\epsilon_n}(\rho_j)$ and normalised (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is $\Sigma_n = s \, [\, \widehat{\text{Cov}}_{\tilde{W}}(A_n) + \lambda I \,]$ with a fixed jitter $\lambda = 10^{-6}$ for positive-definiteness and s the `perturbation_scale`; it is Cholesky-factored once per call. Each mixture component is truncated to the prior box, with its log normalising mass computed analytically so the proposal integrates to one on the support.

Matched pyABC acceptor.

For the synchronous baseline, pyABC's `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy ρ with probability $K_\epsilon(\rho)$ (peak-normalised so $K_\epsilon(0) = 1$), using the *same* kernel K_ϵ as the asynchronous side; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5).

Bounded-memory posterior estimator.

The retroactive AMIS estimator (6) evaluates the snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \leq S = 20$ history-reconstructed proposals with a defensive prior component (mass floored at $0.5/(m{+}1)$) — at each of the n evaluated particles, at total cost $\mathcal{O}(n\,m\,k) = \mathcal{O}(nk)$ with m fixed. The m snapshots are taken draw-proportionally from the reconstructed archive (indices $0, \text{step}, 2\,\text{step}, \ldots$ with step $= \max(1, \lfloor |\text{archive}|/m \rfloor)$). For near-instantaneous simulators the history reaches $\mathcal{O}(10^7)$ particles, so the pass is accumulated in chunks of $2^{16} = 65{,}536$ history points: peak memory is $\mathcal{O}(\text{chunk} \cdot m \cdot k)$ rather than $\mathcal{O}(n)$. Each particle's denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across* chunk boundaries), so chunking changes memory, not the value, and the result is bit-identical to the unchunked computation. Measured on the largest history (Gaussian-mean, $n \approx 3 \times 10^7$ evaluated particles, $k = 100$, $m = 20$), the one-time retroactive pass runs in ≈20 minutes on a single core with ≈0.3 GB peak memory (independent of n; the intermediate is one 2^{16}-point chunk) and parallelizes trivially across evaluation points; it runs after the timed budget and is excluded from the throughput measurements by design. For a near-instantaneous simulator this is comparable to the timed budget itself — non-trivial though bounded — whereas for cost-bearing simulators, whose histories are orders of magnitude shorter, it is negligible against the simulation time.

Appendix B Additional Experimental Protocol

Seeding.

Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`--extend`) runs.

Crash recovery (kill-and-resume).

We test the crash-recoverability claim of §3.1 directly. On the Gaussian-mean benchmark (4 workers, a 1.2×10^5-simulation budget) we compare a clean uninterrupted run against a run that is hard-killed (`SIGKILL` of the entire worker process group) partway through inference and then relaunched with an identical command; recovery reloads the periodic population checkpoint and resumes from the next generation. The resumed run recovers the analytic posterior mean — absolute error 7.6×10^{-3}, against 7.5×10^{-4} for the clean run — and completes the same total simulation budget with no duplicated records. It is *not* a bit-identical replay: the two reported posterior means differ by 6.9×10^{-3}, because the candidate generator is re-seeded from its (replicate, rank) key on restart rather than checkpointed, and MPI arrival order is not deterministic. This is exactly the sense in which the method is "reproducible up to MPI arrival order" (§3.1): recovery yields a *valid* posterior trajectory over the actual evaluated history, not the identical one. In this instance the kill landed *during* a checkpoint write and truncated the primary checkpoint; recovery fell back to the previous-state backup

Table B1 Resolved experiment configurations. Workers, per-run wall-clock budget, replicates (trials for SBC), archive size k, and methods (A: asynchronous; S: matched-kernel synchronous baseline; R: rejection ABC). † Simulation-limited: the budget is per-trial (SBC) or per-run (kill-and-resume) *simulations*, not wall-clock seconds. ‡ Also run with AMIS on/off, a drain-after-deadline variant, and a steeper coupling regime (base delay 0.5 s, cap 8 s).

Experiment	Workers	Budget (s)	Reps	k	Methods
Gaussian mean	48	300	5	100	A,S,R
g-and-k	48	600	5	100	A,S,R
Lotka–Volterra	48	600	5	100	A,S
Cellular Potts	48	3600	5	100	A,S,R
Straggler	16	300	5	100	A,S
Runtime heterogeneity	48	60	5	100	A,S
Parameter-coupled runtime‡	48	60	5	100	A,S
Strong scaling (LV)	1–288	180	5	100/1000	A,S
Strong scaling (CPM)	1–384	1800	3–5	100	A,S
Sensitivity	48	300	5	50/200	A
Ablation	48	300	5	100	A
SBC (Gaussian, 1-D)	48	20k†	1000	100	A,S
SBC (bimodal)	48	20k†	1000	100	A,S
SBC (g-and-k, 4-D)	48	40k†	1000	200	A,S
SBC (g-and-k, 2×)	48	80k†	500	200	A,S
Kill-and-resume	4	120k†	1	100	A

that the dumper writes before overwriting, so the resume succeeded automatically — the reported estimator, being a pure function of the recovered history, is unaffected by which of the two checkpoints is used.

Cluster and wall-clock control.

Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All wall-clock-budgeted methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serialises its current state into the records and returns, so a partial final generation is recorded rather than discarded (the simulation-limited studies marked † in Table B1 — the SBC and kill-and-resume runs — instead stop at a fixed per-trial simulation count). Posterior estimation and plotting run off the timed inference path, so they do not count against the budget.

Benchmark configurations.

Table B1 lists the resolved settings (k is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size $k \in \{100, 1000\}$; all other studies fix the workers shown.

Benchmark-specific notes.

Each benchmark fixes one observed dataset (generated from seed 42) and infers parameters under uniform priors; the discrepancy is a distance between simulated and observed summary statistics. *Gaussian mean:* $n_{\text{obs}} = 100$ observations $\sim \mathcal{N}(0, 1)$, prior $\mu \sim \mathcal{U}[-5, 5]$, summary the sample mean, discrepancy $|\bar{x}_{\text{sim}} - \bar{x}_{\text{obs}}|$; the closed-form Gaussian posterior is the calibration and recovery reference. *g-and-k:* the quantile-defined g-and-k distribution ($c = 0.8$), $n_{\text{obs}} = 1000$, true $(A, B, g, k) = (3, 1, 2, 0.5)$, priors $A \sim \mathcal{U}[0, 6]$, $B \sim \mathcal{U}[0.1, 4]$, $g \sim \mathcal{U}[0, 5]$, $k \sim \mathcal{U}[0, 1]$; summary the seven octiles (the $0.125, \ldots, 0.875$ quantiles), discrepancy Euclidean. *Lotka–Volterra:* a Gillespie stochastic simulation of the four-reaction predator–prey system from $(X_0, Y_0) = (50, 100)$ over $T = 30$, true rates $\theta = (0.5, 0.025, 0.025, 0.5)$, priors $\theta_1, \theta_4 \sim \mathcal{U}[0.05, 2]$ and $\theta_2, \theta_3 \sim \mathcal{U}[0.001, 0.2]$; summary the six statistics $(\overline{X}, \operatorname{sd} X, \overline{Y}, \operatorname{sd} Y, \log(1+\operatorname{var} X/\overline{X}), \log(1+\operatorname{var} Y/\overline{Y}))$, discrepancy Euclidean. *Cellular Potts:* the real `cellsInSilico` (NAStJA) simulator, inferring the cell division rate and motility on normalized $[0, 1]$ priors that map to physical ranges $[6\times10^{-5}, 0.6]$ and $[0, 10^4]$ (true normalized values 0.0499 and 0.2); the discrepancy is a whitened distance over morphological summary features of the simulated cell configuration (radial density and pair-correlation profiles, free-area and structure-factor profiles, a DBSCAN dispersion fraction, and log cell count and radius) against reference summaries. Its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

Computing environment.

All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and ≈94 GB memory per node, connected by Mellanox EDR InfiniBand. The memory-heavy fast-simulator reruns instead used the 180 GB high-memory partition. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT for the sliced Wasserstein distance; the asynchronous method runs as a propagator in our fork of Propulate. Unless noted, ranks are placed one per physical core (`srun --ntasks-per-node=48 --cpus-per-task=1`), so a "W-worker" point occupies W physical cores packed onto $\lceil W/48 \rceil$ fully-reserved nodes; the memory-heavy Gaussian-family reruns instead placed 24 ranks per high-memory node under whole-node reservation. Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

References

[1] Beaumont, M.A.: Approximate bayesian computation. Annual Review of Statistics and Its Application **6**, 379–403 (2019) https://doi.org/10.1146/annurev-statistics-030718-105212

[2] Sisson, S.A., Fan, Y., Tanaka, M.M.: Sequential monte carlo without likelihoods. Proceedings of the National Academy of Sciences **104**(6), 1760–1765 (2007) https://doi.org/10.1073/pnas.0607208104

[3] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems. Journal of the Royal Society Interface **6**(31), 187–202 (2009) https://doi.org/10.1098/rsif.2008.0172

[4] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate bayesian computation. Biometrika **96**(4), 983–990 (2009) https://doi.org/10.1093/biomet/asp052

[5] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential monte carlo method for approximate bayesian computation. Statistics and Computing **22**(5), 1009–1020 (2012) https://doi.org/10.1007/s11222-011-9271-y

[6] Taubert, O., Weißer, M., Kelle, D., Bayer, M., Knüttel, H., Comito, C., Streit, A., Götz, M.: Massively parallel genetic optimization through asynchronous propagation of populations. In: International Conference on High Performance Computing (ISC High Performance), pp. 106–124 (2023). https://doi.org/10.1007/978-3-031-32041-5_6

[7] Wilkinson, R.D.: Approximate bayesian computation (abc) gives exact results under the assumption of model error. Statistical Applications in Genetics and Molecular Biology **12**(2), 129–141 (2013) https://doi.org/10.1515/sagmb-2013-0010

[8] Fearnhead, P., Prangle, D.: Constructing summary statistics for approximate bayesian computation: semi-automatic approximate bayesian computation. Journal of the Royal Statistical Society: Series B **74**(3), 419–474 (2012) https://doi.org/10.1111/j.1467-9868.2011.01010.x

[9] Cornuet, J.-M., Marin, J.-M., Mira, A., Robert, C.P.: Adaptive multiple importance sampling. Scandinavian Journal of Statistics **39**(4), 798–812 (2012) https://doi.org/10.1111/j.1467-9469.2011.00756.x

[10] Klinger, E., Rickert, D., Hasenauer, J.: pyabc: distributed, likelihood-free inference. Bioinformatics **34**(20), 3591–3593 (2018) https://doi.org/10.1093/bioinformatics/bty361

[11] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyabc 0.12.3: high-performance distributed, likelihood-free inference. Bioinformatics **38**(13), 3354–3355 (2022) https://doi.org/10.1093/bioinformatics/btac321

[12] Drovandi, C.C., Pettitt, A.N.: Estimation of parameters for macroparasite population evolution using approximate bayesian computation. Biometrics **67**(1), 225–233 (2011) https://doi.org/10.1111/j.1541-0420.2010.01410.x

[13] Lenormand, M., Jabot, F., Deffuant, G.: Adaptive approximate bayesian computation for complex models. Computational Statistics **28**(6), 2777–2796 (2013)

https://doi.org/10.1007/s00180-013-0428-3

[14] Veach, E., Guibas, L.J.: Optimally combining sampling techniques for monte carlo rendering. In: Proceedings of SIGGRAPH '95, pp. 419–428 (1995). https://doi.org/10.1145/218380.218498

[15] Cappé, O., Guillin, A., Marin, J.-M., Robert, C.P.: Population Monte Carlo. Journal of Computational and Graphical Statistics **13**(4), 907–929 (2004) https://doi.org/10.1198/106186004X12803

[16] Marin, J.-M., Pudlo, P., Sedki, M.: Consistency of adaptive importance sampling and recycling schemes. Bernoulli **25**(3), 1977–1998 (2019) https://doi.org/10.3150/18-BEJ1042

[17] Bugallo, M.F., Elvira, V., Martino, L., Luengo, D., Míguez, J., Djurić, P.M.: Adaptive importance sampling: The past, the present, and the future. IEEE Signal Processing Magazine **34**(4), 60–79 (2017) https://doi.org/10.1109/MSP.2017.2699226

[18] Vergé, C., Dubarry, C., Del Moral, P., Moulines, E.: On parallel implementation of sequential Monte Carlo methods: The island particle model. Statistics and Computing **25**(2), 243–260 (2015) https://doi.org/10.1007/s11222-013-9429-x

[19] Paige, B., Wood, F., Doucet, A., Teh, Y.W.: Asynchronous anytime sequential monte carlo. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 27 (2014)

[20] Murray, L.M., Lee, A., Jacob, P.E.: Parallel resampling in the particle filter. Journal of Computational and Graphical Statistics **25**(3), 789–805 (2016) https://doi.org/10.1080/10618600.2015.1062015

[21] Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., Mira, A.: Abcpy: A high-performance computing perspective to approximate bayesian computation. Journal of Statistical Software **100**(7), 1–38 (2021) https://doi.org/10.18637/jss.v100.i07

[22] Lintusaari, J., Vuollekoski, H., Kangasrääsiö, A., Skytén, K., Järvenpää, M., Marttinen, P., Gutmann, M.U., Vehtari, A., Corander, J., Kaski, S.: ELFI: Engine for likelihood-free inference. Journal of Machine Learning Research **19**(16), 1–7 (2018)